Effective and Efficient Multi-Vector Text Clustering

Hongtao Wang
Hong Kong Baptist University
Hong Kong SAR, China
cshtwang@comp.hkbu.edu.hk

Renchi Yang
Hong Kong Baptist University
Hong Kong SAR, China
renchi@hkbu.edu.hk

Abstract

Text clustering is a fundamental technique for organizing large unlabeled document collections. Existing methods predominantly follow a *single-vector* paradigm, which enables efficient metric clustering but compresses away fine-grained semantics, especially in long and multi-faceted documents. LLM-guided methods can offer richer pairwise judgments, yet repeated model calls impose prohibitive costs at corpus scale. Multi-vector representations preserve multiple constituent-level semantics and offer a promising alternative, yet clustering documents under such set-valued representations effectively and efficiently remains largely unexplored.

This paper presents a systematic study of *multi-vector text clustering*. Among representative multi-vector distances, we identify the *symmetric normalized Chamfer distance* (SNCD) as particularly effective, both empirically and theoretically. However, direct clustering with SNCD incurs quadratic time and space costs due to the need of materializing all pairwise distances, which is impractical. To address this problem, we develop `Chamfer-KMeans`, an exact Lloyd-style algorithm for a new SNCD-based centroid objective. By introducing *Fréchet Mean* for centroid estimation and treating bidirectional nearest-neighbor matches as latent assignments, `Chamfer-KMeans` admits a closed-form centroid update with monotonic descent and convergence guarantees. To overcome the substantial computational cost and sensitivity of `Chamfer-KMeans` to initialization, we further develop `Chamfer-KMeans+`. It capitalizes on cross-block *fixed-dimensional encodings* (FDEs) for fast bounded-error SNCD estimation, lazy cluster assignment, and FDE-based greedy seeding, eliminating most exact SNCD evaluations and improving convergence. Experiments on eight benchmark datasets show that our methods outperform strong single-vector, LLM-guided, and multi-vector baselines in clustering quality. On CPUs, `Chamfer-KMeans+` is up to 21.5× faster than `Chamfer-KMeans` and up to 151.4× faster than pairwise multi-vector baselines, while scaling to 1.46 million texts with comparable clustering quality.

PVLDB Reference Format:
Hongtao Wang and Renchi Yang. Effective and Efficient Multi-Vector Text Clustering. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://anonymous.4open.science/r/MVTC-A34F.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 Introduction

Text clustering [48] is a fundamental primitive for organizing large unlabeled corpora, and underpins a wide range of applications, such as information retrieval over digital libraries or document databases [53, 66], scientific literature analysis [12, 61], enterprise knowledge management [35], and so on, where manual labels are unavailable or prohibitively expensive to maintain. The dominant methodology for text clustering represents each document by a single fixed-dimensional embedding produced by pretrained text encoders (e.g., BERT-style [45, 55]), and then applies a standard clustering method, such as K-`Means` [33], *hierarchical agglomerative clustering* (`HAC`) [68], or *spectral clustering* (`SpecClust`) [63]. This pipeline has gained widespread adoption due to its simple and modular paradigm, the availability of increasingly effective text encoding models, and, equally importantly, the mature and highly optimized metric clustering algorithms developed in the literature.

Recent research [38, 52] has demonstrated that this single-vector paradigm can achieve substantially improved performance when *Large Language Model* (LLM) embeddings are used as document representations. Nevertheless, the resulting clustering quality remains sub-optimal, particularly for long and multi-faceted documents, since a single embedding vector often fails to faithfully capture the nuanced semantics and diverse signals underlying a document [1, 26, 40]. Another line of recent work [23, 51, 60, 62, 65, 77] resorts to cross-attention models (PaLM or GPT), or instruction-tuned LLMs, to directly evaluate the similarities between documents, and then formulate text clustering as a weighted or signed graph clustering problem. Although LLMs can provide accurate semantic similarity judgments and thereby improve clustering quality, such approaches typically require considerable pairwise document comparisons and calls to LLMs, potentially up to $O(N^2)$ for a corpus of N documents, rendering them impractical at corpus scale. Moreover, each model call entails either expensive inference with a large cross-attention model or repeated calls to commercial LLM APIs, leading to substantial computational, latency, and monetary costs, especially when documents are long.

The fundamental limitations of single-vector text representations have already become evident in query-document retrieval, as theoretically analyzed by Google [37, 69] and subsequent studies [1, 67]. Specifically, a fixed-dimensional vector can only realize a restricted family of relevance orderings, even when the downstream task uses simple queries. As a remedy, *late-interaction* retrieval models such as ColBERT [39] and its follow-up works [22, 58] propose to retain multiple contextualized vectors per text and compare queries and documents through fine-grained token-level matches, a mechanism known as late interaction [39]. This design allows the query, usually a short text, to match multiple semantic facets of a document through its keywords, thereby enabling more fine-grained and effective retrieval. Inspired by these observations, a

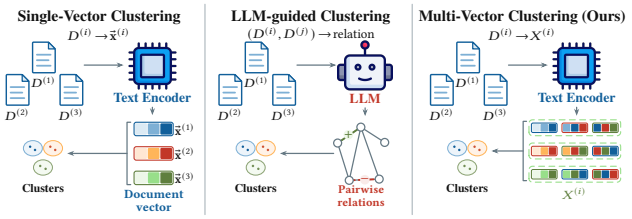


Figure 1: Text Clustering Paradigms

natural idea is to represent each document $D^{(i)}$ as a multi-vector object $\{\mathbf{x}_1^{(i)}, \ldots, \mathbf{x}_{n_i}^{(i)}\}$, since a document is inherently composed of multiple semantic units, such as tokens, sentences, paragraphs, or chunks [73]. However, to our knowledge, how to effectively and efficiently cluster documents under such multi-vector representations remains largely unexplored.

To bridge this gap, this paper conducts a systematic study of text clustering with multi-vector representations. First, we experimentally evaluate the performance of multi-vector clustering based on classic set-to-set distance measures, including *Hausdorff distance* [36], *Wasserstein distance* [56], and *symmetric normalized Chamfer distance* (SNCD) [27], against strong single-vector clustering baselines over diverse document datasets. Our preliminary empirical comparisons (§3.1) reveal the great promise of the multi-vector clustering paradigm, particularly with SNCD. Unfortunately, this direction also presents formidable computational challenges. Exact SNCD calculation introduces a much heavier computational burden than Euclidean distances, while existing clustering algorithms are largely built around single-vector inputs, and hence, ill-suited to the set-valued nature of multi-vector representations.

To achieve effective and computationally practical multi-vector text clustering, we first formulate a new SNCD-based centroid objective and devise an exact Lloyd-style algorithm, `Chamfer-KMeans`, to optimize it. At a high level, we propose to construct the *Fréchet Mean* [24] of multi-vectors as their estimated centroid. Based thereon, we derive a closed-form centroid update and cluster assignment rules using alternating optimization. It can monotonically decrease the objective and converge to a local minimum, without materializing all pairwise distances.

However, our empirical analysis reveals that `Chamfer-KMeans` remains computationally expensive and sensitive to initialization. Each iteration requires $N \times K$ SNCD evaluations for cluster assignment, whilst random initialization can lead to poor local minima and slow, unstable convergence, rendering `Chamfer-KMeans` impractical on large datasets. In response, we further upgrade it to a scalable and robust approximate variant, `Chamfer-KMeans+`, through a suite of carefully-crafted optimization techniques. Specifically, we develop theoretically-grounded *cross-block fixed-dimensional encodings (FDEs)* that reduce expensive SNCD computations to single-vector dot products, enabling fast SNCD estimation. Building on this primitive, we devise a *lazy cluster assignment scheme* that safely prunes *stable* texts under rigorous theoretical guarantees and judiciously leverages cross-block FDEs to identify a handful of candidate centroids. Finally, an FDE-based greedy seeding strategy combines objective- and coverage-guided sampling to improve initialization. Our extensive experiments on eight datasets exhibit that `Chamfer-KMeans` is able to attain consistently superior clustering quality while being considerably faster than pairwise multi-vector baselines. `Chamfer-KMeans+` further accelerates `Chamfer-KMeans` by up to 21.5× on CPUs, and scales to 1.46 million texts, without compromising result quality.

2 Preliminaries

2.1 Symbols and Terminology

Definition 2.1 (Multi-Vector). A multi-vector $X^{(i)}$ is defined as a set of L_2-normalized vectors in $\mathbb{R}^d$, i.e., $\{\vec{\mathbf{x}}_1^{(i)}, \vec{\mathbf{x}}_2^{(i)}, \ldots, \vec{\mathbf{x}}_{n_i}^{(i)}\}$, where $\|\vec{\mathbf{x}}_k^{(i)}\|_2 = 1$ and $n_i = |X^{(i)}|$.

We denote a corpus of N texts by $\{D^{(1)}, D^{(2)}, \ldots, D^{(N)}\}$, wherein each text $D^{(i)}$ is often represented as a d-dimensional embedding vector $\vec{\mathbf{x}}^{(i)} = \mathsf{TextEncoder}(D^{(i)}) \in \mathbb{R}^d$. The text encoder can be a *pre-trained language model* (PLM), e.g., BERT [19] and its successors [45, 55], or an LLM. Using text encoders such as ColBERT [39], SentenceBERT (SBERT) [55], and Jina [31], a text can also be represented by a set or sequence of token-, sentence-, or chunk-level embeddings. For ease of exposition, we refer to this set of embedding vectors for $D^{(i)}$ as a *multi-vector* $X^{(i)}$ defined in Definition 2.1, and each vector $\vec{\mathbf{x}}_k^{(i)}$ in $X^{(i)}$ as a *constituent vector*. The cardinality of multi-vector $X^{(i)}$ is denoted by n_i. For the multi-vector representation of the corpus, we write $\mathcal{D} = \{X^{(1)}, \ldots, X^{(N)}\}$ and denote the maximum multi-vector cardinality by $n_{\max} = \max_{X^{(i)} \in \mathcal{D}} n_i$. In addition, we use $\widehat{N}$ to denote the total number of constituent vectors in $\mathcal{D}$, i.e., $\widehat{N} = \sum_{X^{(i)} \in \mathcal{D}} n_i$.

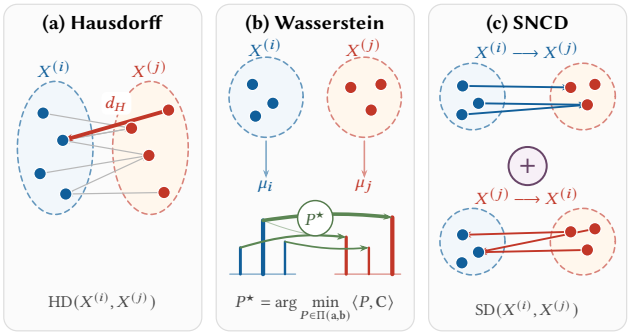


Figure 2: Illustration of multi-vector distance measures.

2.2 Multi-Vector Distance Measures

In what follows, we formally introduce three representative measures for comparing multi-vectors. Figure 2 illustrates their distinct matching principles, while Table 1 summarizes their computational complexities.

Hausdorff Distance. The Hausdorff distance [36] provides a strict set-level comparison by focusing on the worst-case mismatch between two multi-vectors. To be precise, the Hausdorff distance between $X^{(i)}$ and $X^{(j)}$ evaluates the maximum of all nearest-neighbor distances:

$$\mathrm{HD}(X^{(i)}, X^{(j)}) = \max\left\{ \max_{\vec{\mathbf{x}} \in X^{(i)}} \min_{\vec{\mathbf{y}} \in X^{(j)}} \|\vec{\mathbf{x}} - \vec{\mathbf{y}}\|_2, \max_{\vec{\mathbf{y}} \in X^{(j)}} \min_{\vec{\mathbf{x}} \in X^{(i)}} \|\vec{\mathbf{y}} - \vec{\mathbf{x}}\|_2 \right\} \quad (1)$$

It has two inherent deficiencies. First, it is sensitive to noisy, rare, or outlier constituent vectors due to the maximum operator. Second,

it suffers from a quadratic cost as its exact computation requires evaluating all pairwise distances between $X^{(i)}$ and $X^{(j)}$ [78].

Wasserstein Distance. The *Wasserstein distance* (a.k.a. *Earth Mover's Distance*) [56] treats each multi-vector as a discrete distribution and computes an *optimal transport* [16] plan between the two distributions. Let $\mathbf{a} \in \mathbb{R}^{n_i}$ and $\mathbf{b} \in \mathbb{R}^{n_j}$ be the weights assigned to the constituent vectors of $X^{(i)}$ and $X^{(j)}$, respectively, and $\mathbf{C} \in \mathbb{R}^{n_i \times n_j}$ be the ground-cost matrix. The set of feasible transport plans is $\Pi(\mathbf{a}, \mathbf{b}) = \left\{ \mathbf{P} \in \mathbb{R}_+^{n_i \times n_j} \;\middle|\; \mathbf{P}\mathbf{1} = \mathbf{a},\ \mathbf{P}^\top \mathbf{1} = \mathbf{b} \right\}$. Accordingly, the entropy-regularized Wasserstein distance is defined as

$$\mathrm{WD}(X^{(i)}, X^{(j)}) = \min_{\mathbf{P} \in \Pi(\mathbf{a},\mathbf{b})} \left(\sum_{\ell=1}^{n_i} \sum_{k=1}^{n_j} \mathbf{P}_{\ell,k} \mathbf{C}_{\ell,k} + \lambda \sum_{\ell=1}^{n_i} \sum_{k=1}^{n_j} \mathbf{P}_{\ell,k} \log \mathbf{P}_{\ell,k} \right), \quad (2)$$

where $\lambda > 0$ controls the strength of entropy regularization. This regularized objective can be solved using the well-known *Sinkhorn algorithm* [16] with T_s iterations, but the resulting computation can be costly.

Table 1: Comparison of multi-vector distance measures.

Measure	Definition	Complexity
Hausdorff Distance [36]	Eq. (1)	$O(n_i n_j d)$
Wasserstein Distance [56]	Eq. (2)	$O(n_i n_j (d + T_s))$
SNCD [5, 27]	Eq. (3)	$O(n_i n_j d)$

Chamfer Distance and SNCD. For each constituent vector in one multi-vector, *Chamfer distance* [5] (Definition 2.2) finds the closest constituent vector in the other multi-vector and aggregates the resulting matching scores.

Definition 2.2 (Chamfer Distance). Given two multi-vectors $X^{(i)}$ and $X^{(j)}$, their *Chamfer Distance* is defined as $\sum_{\vec{\mathbf{x}} \in X^{(i)}} \min_{\vec{\mathbf{y}} \in X^{(j)}} \|\vec{\mathbf{x}} - \vec{\mathbf{y}}\|_2^2$.

According to Definition 2.2, the standard Chamfer distance is asymmetric, i.e., exchanging its two arguments can change the resulting value. Thus, we consider its natural variant *Symmetric Normalized Chamfer Distance* (SNCD) [27] formulated in the following equation:

$$\mathrm{SD}(X^{(i)}, X^{(j)}) = \sum_{\vec{\mathbf{x}} \in X^{(i)}} \frac{\min_{\vec{\mathbf{y}} \in X^{(j)}} \|\vec{\mathbf{x}} - \vec{\mathbf{y}}\|_2^2}{n_i} + \sum_{\vec{\mathbf{y}} \in X^{(j)}} \frac{\min_{\vec{\mathbf{x}} \in X^{(i)}} \|\vec{\mathbf{y}} - \vec{\mathbf{x}}\|_2^2}{n_j}. \quad (3)$$

Intuitively, SNCD is able to capture more local, fine-grained semantic matches between documents, since it computes the alignment between individual constituent vectors, but leads to serious computational efficiency issues as an aftermath.

2.3 Canonical Clustering Algorithms

Next, we briefly revisit three representative clustering paradigms for text clustering [52]: K-Means [33], hierarchical agglomerative clustering (HAC) [68], and spectral clustering (SpecClust) [63].

K-Means. K-Means is the prototypical centroid-based clustering method, which partitions a dataset into K disjoint clusters by minimizing the WCSS (within-cluster sum of squares, a.k.a. *inertia*). Given a dataset $\{\vec{\mathbf{x}}_1, \ldots, \vec{\mathbf{x}}_N\} \subset \mathbb{R}^d$, the objective is:

$$\min_{\mathcal{P}, \mathcal{M}} \sum_{k=1}^{K} \sum_{\vec{\mathbf{x}}_i \in P_k} \|\vec{\mathbf{x}}_i - \vec{\boldsymbol{\mu}}_k\|_2^2, \quad (4)$$

where $\mathcal{P} = \{P_1, \ldots, P_K\}$ denotes the clusters and $\mathcal{M} = \{\vec{\boldsymbol{\mu}}_1, \ldots, \vec{\boldsymbol{\mu}}_K\}$ are the corresponding cluster centroids.

The standard optimization procedure is Lloyd's algorithm [46], which alternates between: (i) assigning each data point to its nearest centroid, and (ii) updating centroids as the mean of assigned points. This procedure monotonically decreases the objective and converges to a *local* minimum in a finite number of iterations. Each iteration takes $O(NKd)$ time. Despite being efficient, K-Means assumes isotropic, convex clusters and is sensitive to initialization, as studied in K-Means++ [3].

Hierarchical Agglomerative Clustering. HAC constructs a hierarchy of clusters via a *bottom-up* merging process, yielding a dendrogram without requiring a prespecified number of clusters. Unlike K-Means, HAC does not optimize a global objective but instead relies on a linkage function $\mathcal{L}(P_i, P_j)$ that defines inter-cluster distance. Let $\delta(\vec{\mathbf{x}}, \vec{\mathbf{y}})$ denote the point-level dissimilarity between two data points. Common choices [49] include:

- **Single linkage:** $\mathcal{L}(P_i, P_j) = \min_{\vec{\mathbf{x}} \in P_i, \vec{\mathbf{y}} \in P_j} \delta(\vec{\mathbf{x}}, \vec{\mathbf{y}})$;
- **Average linkage:** $\mathcal{L}(P_i, P_j) = \frac{1}{|P_i| \cdot |P_j|} \sum_{\vec{\mathbf{x}} \in P_i} \sum_{\vec{\mathbf{y}} \in P_j} \delta(\vec{\mathbf{x}}, \vec{\mathbf{y}})$;
- **Complete linkage:** $\mathcal{L}(P_i, P_j) = \max_{\vec{\mathbf{x}} \in P_i, \vec{\mathbf{y}} \in P_j} \delta(\vec{\mathbf{x}}, \vec{\mathbf{y}})$.

Specifically, HAC starts from singleton clusters and then repeatedly merges the pair of clusters with the minimum linkage value until K clusters remain. A naive implementation requires $O(N^3)$ time and $O(N^2)$ space, while optimized variants [18, 49, 59] reduce runtime to $O(N^2 \log N)$ or $O(N^2)$ depending on the linkage strategy. HAC is attractive for its flexibility and hierarchical structure, but is computationally expensive and sensitive to linkage choice.

Spectral Clustering. Spectral clustering reformulates clustering as a *graph partitioning problem* to enable manifold structure discovery. Specifically, the weighted graph is represented by an affinity matrix $\mathbf{W} \in \mathbb{R}^{N \times N}$, where the affinity between any two data points $\vec{\mathbf{x}}_i, \vec{\mathbf{x}}_j$ is typically the Gaussian kernel $\exp\left(-\|\vec{\mathbf{x}}_i - \vec{\mathbf{x}}_j\|_2^2 / 2\sigma^2\right)$ [63]. Accordingly, we can define a diagonal degree matrix $\mathbf{D}$ ($\mathbf{D}_{i,i} = \sum_{j=1}^{N} \mathbf{W}_{i,j}$) and the normalized Laplacian matrix $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{W} \mathbf{D}^{-1/2}$.

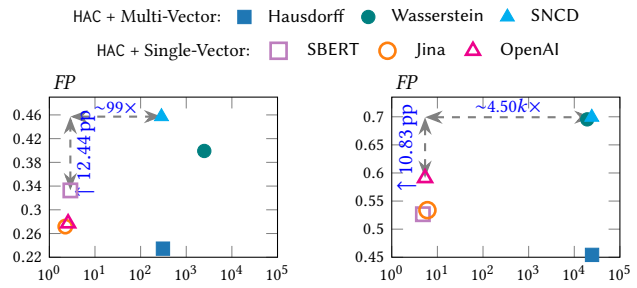
One standard spectral clustering objective is the normalized cut [63], which minimizes inter-cluster connectivity relative to cluster volume. It admits the discrete trace formulation $\min_{\mathbf{H}} \mathrm{Tr}(\mathbf{H}^\top \mathbf{L} \mathbf{H})$, where $\mathbf{H}_{i,k} = \sqrt{\mathbf{D}_{i,i} / \sum_{\vec{\mathbf{x}}_j \in P_k} \mathbf{D}_{j,j}}$ if $\vec{\mathbf{x}}_i \in P_k$ and 0 otherwise. This discrete optimization problem is NP-hard. The standard method leverages partial eigendecomposition for approximate solutions. The main computational bottlenecks lie in graph construction ($O(N^2 d)$) and partial eigendecomposition ($O(N^2 K)$ using iterative methods).

3 Single- vs. Multi-Vector Text Clustering

We next compare multi-vector text clustering with the conventional single-vector paradigm. We first present an empirical study and then discuss the technical challenges introduced by multi-vector representations.

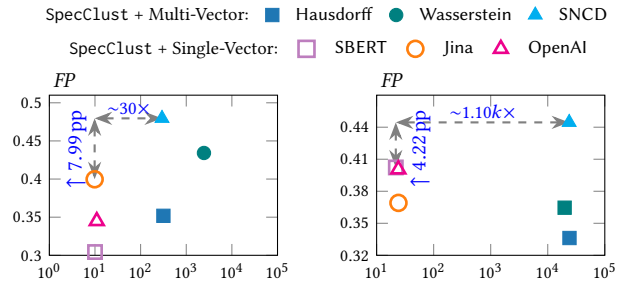
3.1 Empirical Study and Analysis

We first conduct a controlled comparison on *TREC* and *Reuters-52* datasets (see Table 3). The single-vector baselines represent each text using embeddings from SBERT, Jina, and OpenAI, whereas the multi-vector variants retain fine-grained constituent embeddings



(a) *TREC* ($N = 5,952$) (b) *Reuters-52* ($N = 9,100$)

Figure 3: Clustering quality vs. runtime with HAC.



(a) *TREC* ($N = 5,952$) (b) *Reuters-52* ($N = 9,100$)

Figure 4: Clustering quality vs. runtime with SpecClust.

and compare the resulting multi-vectors using Hausdorff, Wasserstein, or SNCD. Following [52], we evaluate both paradigms using HAC and spectral clustering (dubbed as SpecClust). Notice that the off-the-shelf K-Means cannot be directly applied here, since its arithmetic-mean centroid update must be in a Euclidean vector space and is not defined for multi-vectors. HAC directly consumes the resulting distance matrix, while SpecClust transforms it into an affinity matrix before graph partitioning. We report FP [25, 64], the harmonic mean of Purity and Inverse Purity, as the clustering quality measure, together with end-to-end clustering runtime measured in seconds.

Figures 3 and 4 show a consistent quality advantage for multi-vector clustering with SNCD. Particularly, under HAC, SNCD significantly improves FP over the strongest single-vector baseline by 12.44 and 10.83 percentage points on *TREC* and *Reuters-52*, respectively. The corresponding gains under SpecClust are also remarkable, which are 7.99 and 4.22 percentage points. Thus, SNCD achieves the highest text clustering quality in all displayed settings, despite being compared against the best result among three strong single-vector encoders. These empirical results suggest that the improvement primarily stems from preserving and matching fine-grained text semantics rather than from a particular clustering algorithm. More specifically, retaining multiple vectors alone does not guarantee better clustering. Hausdorff distance performs worse than the strongest single-vector baseline in all settings, while Wasserstein distance is less consistently effective compared to SNCD. This observation highlights that the aggregation of constituent-level semantics is as important as the representation itself. SNCD averages bidirectional nearest-neighbor matches, which allows related semantic facets to match locally without being dominated by either a single outlier or a globally balanced transport constraint.

However, the quality gains are obtained at a substantial computational cost. Compared with the strongest single-vector baseline in terms of FP, exact SNCD is approximately 99× and 4,500× slower under HAC, and 30× and 1,100× slower under SpecClust. The reason is that computing SNCD between two multi-vectors requires comparing their constituent vectors, taking $\Theta(n_i n_j d)$ time, while both HAC and SpecClust require pairwise relationships among all N texts. We additionally conduct a theoretical comparison of both clustering paradigms in the supplementary material [4].

3.2 Technical Challenges

The preceding study reveals a clear effectiveness-scalability gap: multi-vector clustering with SNCD consistently improves text clustering quality, but incurs tens to thousands of times more runtime than single-vector clustering. Computing SNCD between two multi-vectors takes $\Theta(n_i n_j d)$ time, while HAC and SpecClust require $\Theta(N^2)$ pairwise text comparisons. Their combination is therefore prohibitively expensive for large corpora, particularly when individual texts contain many constituent vectors. Single-vector clustering avoids all-pairs computation through K-Means, which scales almost linearly with corpus size N for fixed K. However, extending this paradigm to multi-vectors raises three challenges:

- How can we define and efficiently update a multi-vector centroid under SNCD, for which the arithmetic-mean update in classic K-Means is no longer applicable?
- Although a Lloyd-style procedure reduces N^2 text pairs to NK text–centroid pairs, each SNCD evaluation remains costly. How can we avoid redundant exact evaluations and accelerate the remaining constituent-level comparisons?
- How can we obtain representative initial centroids without costly corpus-wide SNCD computation, thereby improving result quality and convergence behavior?

These challenges motivate the design and development of our scalable Chamfer-KMeans algorithm in the succeeding sections.

4 The Chamfer-KMeans Algorithm

This section develops Chamfer-KMeans, a Lloyd-style algorithm for clustering multi-vectors under exact SNCD. We first formulate the SNCD-based clustering objective and provide an overview of our solution in Section 4.1. Subsequently, we present our exact base algorithm in Section 4.2 and establish its convergence and time complexity in Section 4.3.

4.1 Basic Idea and Synoptic Overview

Recall that the classic K-Means algorithm is to minimize the within-cluster variance in Eq. (4), which achieves an almost linear time complexity by merely considering point-centroid distances. In a similar vein, we extend the clustering objective in Eq. (4) to multi-vectors, leading to an optimization function:

$$\min_{\mathcal{P},\mathcal{M}} \sum_{k=1}^{K} \sum_{X \in P_k} \mathrm{SD}(X, M_k) \tag{5}$$

where $\mathrm{SD}(X, M_k)$ stands for the SNCD between multi-vector X and centroid M_k, defined in Eq. (3). However, the optimization of the objective function in Eq. (5) fundamentally diverges from standard K-Means due to two major challenges:

Algorithm 1: The Exact Chamfer-KMeans

Input: A set $\mathcal{D}$ of multi-vectors, #clusters K, #iterations T.
Output: Cluster assignments $\mathcal{P} = \{P_1, \ldots, P_K\}$.

1 Randomly select K multi-vectors from $\mathcal{D}$ as $\mathcal{M}$;
2 **for** $t = 1$ **to** T **do**
  /* Cluster Assignment Step */
3 $P_k \leftarrow \emptyset \ \forall P_k \in \mathcal{P}$;
4 **foreach** $X^{(i)} \in \mathcal{D}$ **do**
5 Compute $\mathrm{SD}(X^{(i)}, M_k) \ \forall k \in \{1, \ldots, K\}$; ▷ Eq. (3)
6 $k_i \leftarrow \arg\min_{1 \le k \le K} \mathrm{SD}(X^{(i)}, M_k)$ and add $X^{(i)}$ to P_{k_i};
  /* Centroid Update Step */
7 **foreach** $P_k \in \mathcal{P}$ **do**
8 **foreach** $X \in P_k$ **do**
9 Compute $\eta(\vec{\mathbf{x}}_i, M_k) \ \forall \vec{\mathbf{x}}_i \in X$; ▷ Eq. (6)
10 Compute $\eta(\vec{\boldsymbol{\mu}}_j^{(k)}, X) \ \forall \vec{\boldsymbol{\mu}}_j^{(k)} \in M_k$; ▷ Eq. (7)
11 Update $\vec{\boldsymbol{\mu}}_j^{(k)} \in M_k$; ▷ Eq. (8)
12 **if** $\mathcal{M}$ *remains almost unchanged* **then break**;
13 **return** $\mathcal{P}$;

- Unlike single-vector K-Means which trivially computes an arithmetic mean in a continuous space, finding centroids $\mathcal{M}$ minimizing Eq. (5) requires solving for a complex set-valued *Fréchet Mean* [24] rather than a simple spatial average.
- Standard K-Means optimizes the smooth, convex squared Euclidean distance, whilst the nested min operators render Eq.(5) strictly non-differentiable.

To surmount these challenges, instead of using direct differentiation, our idea is to formulate the centroid calculation as an *alternating optimization process* [6]. By introducing the notion of *multi-vector Fréchet Mean*, we temporarily freeze the constituent vector-level hard assignments via nearest-neighbor mappings, thereby bypassing the non-differentiability introduced by the min operators and successfully deriving a tractable, closed-form update rule for centroids. We refer interested readers to the supplementary material [4] for the detailed derivative steps for the rules of updating the centroids $\mathcal{M} = \{M_k\}_{k=1}^K$ and the cluster assignments $\mathcal{P}$ for the multi-vectors. In what follows, we elaborate on our exact algorithm based on the afore-derived updating rules.

4.2 The Exact Algorithm

We illustrate the exact Chamfer-KMeans algorithm for clustering multi-vectors under the objective in Eq. (5) in Algorithm 1. It takes as input the multi-vector representation of a text collection, i.e., $\mathcal{D} = \{X^{(1)}, \ldots, X^{(N)}\}$, the specified number K of clusters, and the maximum number T of iterations. Then, it randomly picks K multi-vectors from $\mathcal{D}$ as initial centroids $\mathcal{M} = \{M_k\}_{k=1}^K$ at Line 1, each of which is essentially a multi-vector $M_k = \{\vec{\boldsymbol{\mu}}_1^{(k)}, \vec{\boldsymbol{\mu}}_2^{(k)}, \cdots, \vec{\boldsymbol{\mu}}_{|M_k|}^{(k)}\}$.

Subsequently, Algorithm 1 starts an iterative process (T iterations) to alternately update the assignments and centroids, as in traditional K-Means. In the cluster assignment step, we first initialize K empty clusters $\mathcal{P} = \{P_1, \ldots, P_K\}$ at Line 3, each of which corresponds to a centroid M_k in $\mathcal{M}$. For each multi-vector X, we calculate its SNCD values to the K centroids, and assign it to the cluster $\mathcal{P}_{k^*}$ that its nearest centroid M_{k^*} corresponds to (Lines 4-6). Afterwards, Chamfer-KMeans turns to handling each cluster $P_k \in \mathcal{P}$ sequentially and updating its centroid (Lines 7-11). Specifically, for each multi-vector X in the current cluster P_k, we conduct both *forward* and *backward* matching. In the forward matching, for every constituent vector $\vec{\mathbf{x}}_i$ in $X = \{\vec{\mathbf{x}}_1, \vec{\mathbf{x}}_2, \cdots, \vec{\mathbf{x}}_{|X|}\}$, we find its nearest neighbor vector $\eta(\vec{\mathbf{x}}_i, M_k)$ in M_k at Line 9:

$$\eta(\vec{\mathbf{x}}_i, M_k) \leftarrow \arg\min_{\vec{\boldsymbol{\mu}}_j^{(k)} \in M_k} \|\vec{\mathbf{x}}_i - \vec{\boldsymbol{\mu}}_j^{(k)}\|_2. \tag{6}$$

Conversely, for each constituent vector $\vec{\boldsymbol{\mu}}_j^{(k)}$ in M_k, backward matching (Line 10) identifies $\vec{\boldsymbol{\mu}}_j^{(k)}$'s nearest neighbor $\eta(\vec{\boldsymbol{\mu}}_j^{(k)}, X)$ in X by

$$\eta(\vec{\boldsymbol{\mu}}_j^{(k)}, X) \leftarrow \arg\min_{\vec{\mathbf{x}}_i \in X} \|\vec{\boldsymbol{\mu}}_j^{(k)} - \vec{\mathbf{x}}_i\|_2. \tag{7}$$

After gathering all the matches, Algorithm 1 updates each vector $\vec{\boldsymbol{\mu}}_j^{(k)}$ of centroid M_k at Line 11 as follows:

$$\vec{\boldsymbol{\mu}}_j^{(k)} = \frac{\sum_{X \in P_k} \left(\frac{\eta(\vec{\boldsymbol{\mu}}_j^{(k)}, X)}{|M_k|} + \sum_{\vec{\mathbf{x}}_i \in X, \eta(\vec{\mathbf{x}}_i, M_k) = \vec{\boldsymbol{\mu}}_j^{(k)}} \frac{\vec{\mathbf{x}}_i}{|X|} \right)}{\sum_{X \in P_k} \left(\frac{1}{|M_k|} + \frac{|\{\eta(\vec{\mathbf{x}}_i, M_k) = \vec{\boldsymbol{\mu}}_j^{(k)}\}_{\vec{\mathbf{x}}_i \in X}|}{|X|} \right)}. \tag{8}$$

Note that an additional L_2 normalization will be applied to $\vec{\boldsymbol{\mu}}_j^{(k)}$ afterwards such that it is of unit length. Notice that, for higher practical efficiency, we evaluate Eqs. (6) and (7) jointly using batched pairwise-distance matrices over packed tensors. Row- and column-wise reductions recover both directional mappings from the same matrix, followed by vectorized scatter aggregation for Eq. (8). This implementation ensures exactness while avoiding redundant distance computation and per-vector loops on both CPUs and GPUs.

Chamfer-KMeans repeats the above alternating steps until the maximum number T of iterations is reached, or $\mathcal{M}$ remains almost invariant (i.e., has converged; Line 12). To be precise, we monitor the maximum displacement for each centroid M_k across iterations, defined as $\Delta_t = \max_{1 \le k \le K} \mathrm{SD}\left(M_k^{(t-1)}, M_k^{(t)}\right)$, and terminate when $\Delta_t < \varepsilon$ for two consecutive iterations.

Table 2: Runtime breakdown.

Step	Operation	Sub-Cost	Total Cost
Initialization (Line 1)	-	-	$O(Kdn_{\max})$
Cluster Assignment (Lines 3-6)	Line 5	$O(n_i \cdot \lvert M_k \rvert \cdot d)$	$O(\widehat{N}Kdn_{\max})$
	Line 6	$O(K)$	
Centroid Update (Lines 7-11)	Lines 9–10	$O(n_i \cdot \lvert M_k \rvert \cdot d)$	$O(\widehat{N}dn_{\max})$
	Line 11	$O((n_i + \lvert M_k \rvert) \cdot d)$	

4.3 Analyses

Convergence Analysis. We can conceptualize the optimization of Eq. (5) within an alternating minimization framework and establish the convergence of Algorithm 1 in Theorem 4.1. The proof can be done by showing the monotonicity of the cluster assignment and centroid update steps.

Theorem 4.1. *The clustering loss in Eq. (5) is monotonically decreasing as T grows in Algorithm 1, and is mathematically guaranteed to converge to a local minimum in a finite number T of iterations.*

Complexity Analysis. We first consider the time complexity per iteration (Lines 3-12). For the assignment step, $O(n_i \cdot |M_k| \cdot d)$ time is needed for each multi-vector–centroid pair $(X^{(i)}, M_k)$. Consequently, the time cost for all the $N \times K$ pairs is bounded by

$$\sum_{X^{(i)} \in \mathcal{D}} \sum_{M_k \in \mathcal{M}} n_i \cdot |M_k| \cdot d = \widehat{N} d \cdot \sum_{M_k \in \mathcal{M}} |M_k| \leq \widehat{N} K d \cdot n_{\max}.$$

At the centroid update step, the calculations in Eq. (6) and Eq. (7) consume $O(|M_k| \cdot d)$ and $O(n_i \cdot d)$ time, respectively, and thus, require $O(n_i |M_k| d)$ time for all forward and backward mappings per pair (Lines 9-10). Once these mappings are available, Line 11 (Eq. (8)) only performs weighted accumulation over the matched constituent vectors, requiring $O((n_i + |M_k|) \cdot d)$ time per multi-vector. Therefore, the nearest-neighbor computation dominates the centroid update, whose total cost is bounded by $\sum_{k=1}^{K} \sum_{X \in P_k} O(n_i \cdot |M_k| \cdot d) \leq O(\widehat{N} d \cdot n_{\max})$. Table 2 summarizes the computational complexities of three stages in `Chamfer-KMeans`. Considering all the T iterations, the overall time complexity of Algorithm 1 is bounded by $O(T \widehat{N} K d n_{\max})$.

As for space overhead, aside from the input set of multi-vectors $\mathcal{D}$ that consumes $O(\widehat{N} d)$ space, Algorithm 1 needs to maintain N cluster assignments in $\mathcal{P}$ and K centroids $\mathcal{M}$ that require at most $O(K d n_{\max})$ space. In sum, the total space complexity is bounded by $O((\widehat{N} + K n_{\max}) \cdot d + N)$.

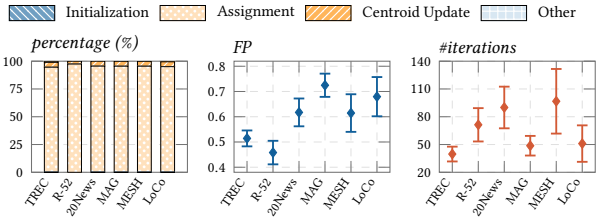
5 Optimizations

5.1 Motivation and Overview

Despite avoiding all-pairs multi-vector comparisons required by `HAC` and `SpecClust`, exact `Chamfer-KMeans` remains prohibitively expensive even on modest corpora. It takes 207 seconds (3.5 minutes) to process *TREC* with only 5,952 texts and 26,638 seconds (444 minutes) for *Reuters-52* (*R-52*) with 9,100 texts using CPUs (Figure 6), rendering it impractical on large text collections. Table 2 identifies cluster assignment as the major computational bottleneck: it evaluates exact SNCD for all $N \times K$ multi-vector–centroid pairs and incurs an additional factor of K over centroid update. In implementation, each SNCD evaluation computes a constituent-level pairwise-distance matrix and performs bidirectional nearest-neighbor reductions. Batching improves hardware utilization but cannot eliminate these expensive exact comparisons. As confirmed in Figure 5(a), cluster assignment steps account for 94.6-97.5% of the total CPU time.

On top of that, exact `Chamfer-KMeans` is also sensitive to random initialization. As shown in Figure 5(b), the standard deviation of FP reaches up to 7.77 percentage points on LoCoV1. This variation arises because the non-convex alternating optimization used in Algorithm 1 can converge to different local minima. Similarly, its convergence speed is also unstable. Figure 5(c) reports 97 ± 35 iterations on *MESH*, indicating that unfavorable initializations can substantially increase the clustering runtime.

To address these limitations, we develop an approximate variant `Chamfer-KMeans+` of exact `Chamfer-KMeans`, through three complementary techniques. Before delving into their algorithmic details, we first outline how it upgrades exact `Chamfer-KMeans` while



(a) Runtime breakdown (b) Cluster quality variance (c) Variance of #iterations

Figure 5: Empirical study of `Chamfer-KMeans`.

retaining its overall workflow. Specifically, `Chamfer-KMeans+` modifies two steps of Algorithm 1: centroid initialization (Line 1) and cluster assignment (Lines 3-6).

At the heart of our design is a fast surrogate for the expensive SNCD computations required by both optimized steps. By leveraging the theoretically-grounded *fixed-dimensional encoding* (FDE) machinery, we derive a fast inner-product estimator for SNCD in Section 5.2. Building on this primitive, Section 5.3 develops a filter-and-refine scheme that lazily updates cluster assignments, while Section 5.4 further capitalizes on the FDEs to design an effective yet efficient greedy seeding strategy. Finally, Section 5.5 analyzes the time and space complexity of `Chamfer-KMeans+`.

5.2 Fast SNCD Approximation via FDEs

Firstly, we introduce the mathematical apparatus of *fixed-dimensional encoding* (FDE) proposed by `MUVERA` [20] for Chamfer similarity approximation. For a multi-vector X and a target dimension d', `MUVERA` maps X into a source encoding $\vec{\mathbf{f}}_s(X) \in \mathbb{R}^{d'}$ and a target encoding $\vec{\mathbf{f}}_t(X) \in \mathbb{R}^{d'}$, respectively, such that for any two multi-vectors X and Y, we have

$$\widehat{s}(X, Y) = \langle \vec{\mathbf{f}}_s(X), \vec{\mathbf{f}}_t(Y) \rangle \approx s(X, Y), \quad s(X, Y) = \frac{1}{|X|} \sum_{\vec{\mathbf{x}} \in X} \max_{\vec{\mathbf{y}} \in Y} \langle \vec{\mathbf{x}}, \vec{\mathbf{y}} \rangle.$$

Theorem 2.1 by Dhulipala et al. [20] formally established an approximation guarantee $|\widehat{s}(X, Y) - s(X, Y)| \leq \epsilon$ with probability at least $1 - p_f$ with certain parameter settings for randomized mappings used in `MUVERA`, where $s(X, Y)$ stands for the *normalized Chamfer similarity* and ϵ denotes the additive error threshold.

Next, we can readily leverage the foregoing FDEs to approximate the forward and backward terms $\frac{1}{n_i} \sum_{\vec{\mathbf{x}} \in X^{(i)}} \min_{\vec{\mathbf{y}} \in X^{(j)}} \|\vec{\mathbf{x}} - \vec{\mathbf{y}}\|_2^2$ and $\frac{1}{n_j} \sum_{\vec{\mathbf{y}} \in X^{(j)}} \min_{\vec{\mathbf{x}} \in X^{(i)}} \|\vec{\mathbf{y}} - \vec{\mathbf{x}}\|_2^2$ in SNCD $\mathrm{SD}(X^{(i)}, X^{(j)})$ using the fact that all constituent vectors are L_2-normalized. More specifically, we can transform the forward term into its equivalent form:

$$\begin{aligned} \frac{1}{n_i} \sum_{\vec{\mathbf{x}} \in X^{(i)}} \min_{\vec{\mathbf{y}} \in X^{(j)}} \|\vec{\mathbf{x}} - \vec{\mathbf{y}}\|_2^2 &= \frac{1}{n_i} \sum_{\vec{\mathbf{x}} \in X^{(i)}} \min_{\vec{\mathbf{y}} \in X^{(j)}} (2 - 2\langle \vec{\mathbf{x}}, \vec{\mathbf{y}} \rangle) \\ &= 2 - 2s(X^{(i)}, X^{(j)}). \end{aligned}$$

The backward term is obtained by swapping the two objects. Consequently,

$$\mathrm{SD}(X^{(i)}, X^{(j)}) = 4 - 2 \cdot (s(X^{(i)}, X^{(j)}) + s(X^{(j)}, X^{(i)})), \quad (9)$$

which can be approximated by FDEs of $X^{(i)}$ and $X^{(j)}$ as follows:

$$\begin{aligned} \mathrm{SD}(X^{(i)}, X^{(j)}) &\approx 4 - 2\left(\widehat{s}(X^{(i)}, X^{(j)}) + \widehat{s}(X^{(j)}, X^{(i)})\right) \\ &= 4 - 2 \begin{bmatrix} \vec{\mathbf{f}}_s(X^{(i)}) \\ \vec{\mathbf{f}}_t(X^{(i)}) \end{bmatrix}^{\top} \cdot \begin{bmatrix} \vec{\mathbf{f}}_t(X^{(j)}) \\ \vec{\mathbf{f}}_s(X^{(j)}) \end{bmatrix} = \widehat{\mathrm{SD}}(X^{(i)}, X^{(j)}). \end{aligned}$$

Algorithm 2: Cross-Block FDE Construction

Input: Multi-vector X and block dimension d_f.
Output: Cross-block FDEs $\vec{\mathbf{g}}_{\mathrm{L}}(X), \vec{\mathbf{g}}_{\mathrm{R}}(X) \in \mathbb{R}^{2d_f}$.

1 $\vec{\mathbf{f}}_s(X), \vec{\mathbf{f}}_t(X) \leftarrow \mathtt{MUVERA}(X)$;
2 Generate Count-Sketch matrices $\mathbf{S}_0, \mathbf{S}_1 \in \{-1, 0, 1\}^{d_f \times d'}$;
3 $\vec{\mathbf{g}}_{\mathrm{L}}(X) \leftarrow (\mathbf{S}_0 \vec{\mathbf{f}}_s(X)) \,\|\, (\mathbf{S}_1 \vec{\mathbf{f}}_t(X))$;
4 $\vec{\mathbf{g}}_{\mathrm{R}}(X) \leftarrow (\mathbf{S}_0 \vec{\mathbf{f}}_t(X)) \,\|\, (\mathbf{S}_1 \vec{\mathbf{f}}_s(X))$;
5 **return** $\vec{\mathbf{g}}_{\mathrm{L}}(X), \vec{\mathbf{g}}_{\mathrm{R}}(X)$;

As such, we can transform each multi-vector $X^{(i)}$ into two FDEs $\vec{\mathbf{f}}_s(X^{(i)})$ and $\vec{\mathbf{f}}_t(X^{(i)})$ for faster SNCD approximation.

However, directly using the above FDEs is prohibitive, as the raw FDE vectors are too large (d' can be up to tens of thousands of dimensions) to cache and costly for inner product calculation. Although a shared projection (e.g., *Random Projection* [7] or *Count-Sketch* [13]) can be leveraged for dimensionality reduction, the two swapped cross-products are identical copies after concatenation, and averaging them fails to provide variance reduction. Our idea is to apply two independent Count-Sketch maps over raw FDEs to reduce variance, as illustrated in Algorithm 2. Below, we describe its algorithmic details and offer an approximation analysis.

Algorithm and Analysis. Algorithm 2 displays the pseudo-code of our cross-block construction of FDEs. It first invokes the standard MUVERA routine, deferred to the supplementary material [4], to obtain the raw FDEs $\vec{\mathbf{f}}_s(X)$ and $\vec{\mathbf{f}}_t(X)$ for multi-vector X (Line 1). Afterwards, line 2 generates two independent Count-Sketch matrices $\mathbf{S}_0$ and $\mathbf{S}_1 \in \{-1, 0, 1\}^{d_f \times d'}$ by Clarkson and Woodruff [13], each with one randomly signed nonzero per column. They are generated once and reused for every text and centroid. Next, we apply them to raw FDEs $\vec{\mathbf{f}}_s(X)$ and $\vec{\mathbf{f}}_t(X)$ to get two cross-block FDEs $\vec{\mathbf{g}}_{\mathrm{L}}(X)$ and $\vec{\mathbf{g}}_{\mathrm{R}}(X)$ as in Lines 3-4, where $\|$ denotes concatenation. We symmetrize the two cross orientations:

$$\begin{aligned}\begin{bmatrix}\vec{\mathbf{g}}_{\mathrm{L}}(X)\\ \vec{\mathbf{g}}_{\mathrm{R}}(X)\end{bmatrix}^{\top} \cdot \begin{bmatrix}\vec{\mathbf{g}}_{\mathrm{R}}(Y)\\ \vec{\mathbf{g}}_{\mathrm{L}}(Y)\end{bmatrix} &= \sum_{t=0}^{1}\left(\langle \mathbf{S}_t \vec{\mathbf{f}}_s(X), \mathbf{S}_t \vec{\mathbf{f}}_t(Y)\rangle + \langle \mathbf{S}_t \vec{\mathbf{f}}_t(X), \mathbf{S}_t \vec{\mathbf{f}}_s(Y)\rangle\right)\\ &= 2\left(\widehat{s}(X,Y) + \widehat{s}(Y,X)\right) \end{aligned} \tag{10}$$

Thus, each directed similarity is estimated under both sketches and our SNCD surrogate becomes

$$\widehat{\mathrm{SD}}(X,Y) = 4 - \begin{bmatrix}\vec{\mathbf{g}}_{\mathrm{L}}(X)\\ \vec{\mathbf{g}}_{\mathrm{R}}(X)\end{bmatrix}^{\top} \cdot \begin{bmatrix}\vec{\mathbf{g}}_{\mathrm{R}}(Y)\\ \vec{\mathbf{g}}_{\mathrm{L}}(Y)\end{bmatrix}. \tag{11}$$

The two sketches are not required for unbiasedness. Their purpose is to provide two independent replicas of each directed estimate after symmetrization. At a fixed block width d_f, this trick halves the conservative Count-Sketch variance bound compared with reusing one sketch, at the expense of doubling the compressed storage, i.e., we need to cache two $2d_f$-dimensional vectors (cross-block FDEs) per multi-vector object. Lemma 5.1 establishes the approximation bound for Algorithm 2.

Lemma 5.1. *Let* $\Lambda_{X,Y} = \|\vec{\mathbf{f}}_s(X)\|_2\|\vec{\mathbf{f}}_t(Y)\|_2 + \|\vec{\mathbf{f}}_t(X)\|_2\|\vec{\mathbf{f}}_s(Y)\|_2$. *Then, given* X, Y, *for any* $\delta > 0$, *with probability at least* $1 - 2p_f - \delta$, $\widehat{\mathrm{SD}}(X,Y)$ *in Eq.* (11) *satisfies* $\left|\widehat{\mathrm{SD}}(X,Y) - \mathrm{SD}(X,Y)\right| \le 4\epsilon + \frac{2\Lambda_{X,Y}}{\sqrt{d_f \delta}}$.

Following the FDE-construction bound [20], Algorithm 2 constructs the cross-block FDEs of a multi-vector X in $O(|X|d + d')$ time under a fixed FDE configuration, since the sparse Count-Sketch projection adds only $O(d')$ cost. Hence, constructing the reusable cross-block FDEs for the whole corpus once costs $O(\widehat{N}d + Nd')$.

Algorithm 3: Lazy Cluster Assignment

1 Invoke Algorithm 2 to get $\vec{\mathbf{g}}_{\mathrm{L}}(M_k), \vec{\mathbf{g}}_{\mathrm{R}}(M_k)$ for $k = 1, \ldots, K$;
2 **for** $k \leftarrow 1$ **to** K **do**
3 Calculate movement bound B_k by Eq. (12);
4 $P_k \leftarrow \emptyset \ \forall P_k \in \mathcal{P}$;
5 **for** $X^{(i)} \in \mathcal{D}$ **do**
6 **if** *Eq.* (13) *holds* **then** Add $X^{(i)}$ to P_{k_i} ;
7 **else**
8 $\mathcal{A}_i \leftarrow \mathrm{Top}\kappa_{k \neq k_i}\, \widehat{\mathrm{SD}}(X^{(i)}, M_k)$;
9 $k_i \leftarrow \arg\min_{k \in \mathcal{A}_i \cup \{k_i\}} \mathrm{SD}(X^{(i)}, M_k)$;
10 Add $X^{(i)}$ to P_{k_i};

5.3 Lazy Cluster Assignment

Our high-level idea is to *lazily* update cluster assignments through a *filter-and-refine* framework. We regard a text as *stable* if its previously assigned centroid can be certified to remain nearest under the current centroids; otherwise, we compute exact SNCD only over a small set of candidate centroids. Specifically, for a stable text, we certify its previous centroid using cached exact distances and *centroid-movement* bounds, thereby avoiding comparisons with the other current centroids. For a text that cannot be certified, we avoid an exhaustive exact scan by using the cross-block FDEs obtained in Section 5.2 to retrieve a small set of candidate centroids, before evaluating the exact SNCD only for these candidates and assigning this text to its best candidate.

Algorithm. Let $\mathcal{M} = \{M_k\}_{k=1}^{K}$ be the current centroids, which have been updated by Lines 7-10 in Algorithm 1. We call the version of M_k before the centroid update its *reference state*, denoted by M_k^{ref}. Let k_i be the index of the centroid assigned to $X^{(i)}$ in the preceding iteration. Note that this assignment is computed based on its reference state $M_{k_i}^{\mathrm{ref}}$ in the preceding iteration.

Algorithm 3 proceeds in three stages. In the first stage, Line 1 constructs the cross-block FDEs of the current centroids. Note that the corpus FDEs are computed once and reused across iterations. For each centroid M_k, Lines 2-3 compute a movement bound B_k on the change from its reference state M_k^{ref} to its current state M_k, where B_k can be computed by

$$B_k \triangleq \max_{1 \le j \le |M_k|} b_j + \frac{1}{|M_k|}\sum_{j=1}^{|M_k|} b_j, \tag{12}$$

where $b_j = 2\|\vec{\boldsymbol{\mu}}_j^{(k)} - \vec{\mathbf{u}}_j^{(k)}\|_2$ and $\vec{\boldsymbol{\mu}}_j^{(k)}$(resp. $\vec{\mathbf{u}}_j^{(k)}$) stands for the constituent vector in M_k^{ref} (resp. M_k). Then, Line 4 initializes the new clusters P_k.

Lemma 5.2. *If Eq.* (13) *holds, then* M_{k_i} *is the nearest centroid of* $X^{(i)}$, *i.e.,* $k_i = \arg\min_{1 \le k \le K} \mathrm{SD}(X^{(i)}, M_k)$.

After that, Algorithm 3 processes each multi-vector $X^{(i)}$ in $\mathcal{D}$. The second stage is to prune stable texts. More concretely, Line 6 evaluates $\mathrm{SD}(X^{(i)}, M_{k_i})$, its exact distance to the current version of its previously assigned centroid. Line 7 compares this value with

Algorithm 4: FDE-based Greedy Seeding

Input: Multi-vector collection $\mathcal{D}$ and number of clusters K.
Output: Initial centroids $\mathcal{M}^{(0)}$.

1 Invoke Algorithm 2 to get $\vec{\mathbf{g}}_{\mathrm{L}}(X), \vec{\mathbf{g}}_{\mathrm{R}}(X)\ \forall X \in \mathcal{D}$;
2 Pick a random X from $\mathcal{D}$ and set $\mathcal{M}^{(0)} \leftarrow \{X\}$;
3 $\tau \leftarrow 2 + \lfloor \ln K \rfloor$;
4 **while** $|\mathcal{M}^{(0)}| < K$ **do**
5 Sample τ candidates $\mathcal{S}_{\mathrm{obj}}$ as per Eq. (15);
6 Sample τ candidates $\mathcal{S}_{\mathrm{cov}}$ as per Eq. (16);
7 Choose $M \in \mathcal{S}_{\mathrm{obj}} \cup \mathcal{S}_{\mathrm{cov}}$ that minimizes Eq. (14);
8 $\mathcal{M}^{(0)} \leftarrow \mathcal{M}^{(0)} \cup \{M\}$;
9 **return** $\mathcal{M}^{(0)}$;

$\mathrm{SD}(X^{(i)}, M_k^{\mathrm{ref}}) - B_k$, a lower bound on the current exact distance to every alternative centroid M_k. If the certificate holds, i.e.,

$$\mathrm{SD}(X^{(i)}, M_{k_i}) \le \min_{k \neq k_i} \left(\mathrm{SD}(X^{(i)}, M_k^{\mathrm{ref}}) - B_k \right), \tag{13}$$

$X^{(i)}$ retains assignment k_i without any further exact SNCD computation. Lemma 5.2 proves that this pruning is safe. If $X^{(i)}$ cannot be filtered out, Algorithm 3 enters into the third stage (Lines 8-10). Let $\mathcal{A}_i$ denote the κ ($\kappa \ll K$) alternative centroids with the smallest approximate SNCD $\widehat{\mathrm{SD}}(X^{(i)}, M_k)$ defined in Eq. (11) with FDEs. The old centroid M_{k_i} is excluded from $\mathcal{A}_i$ because its current exact distance $\mathrm{SD}(X^{(i)}, M_{k_i})$ is already available at Line 6. Line 9 evaluates exact SNCD only over $\mathcal{A}_i \cup \{k_i\}$ and selects the best centroid.

As current centroids drift farther from their reference states with iterative centroid updates, the movement bounds loosen and fewer texts satisfy the stability certificate. When the fraction of certified texts falls below an update threshold ρ (typically 0.3), the next iteration performs a full update of cluster assignments, recomputing and caching all $N \times K$ exact distances and replacing M_k^{ref} and $\mathrm{SD}(X^{(i)}, M_k^{\mathrm{ref}})$ with the current states. It is worth mentioning that, in each SNCD evaluation, we resort to a fast dot-product kernel based on the fact of $\|\vec{\mathbf{x}} - \vec{\boldsymbol{\mu}}\|_2^2 = \|\vec{\mathbf{x}}\|_2^2 + \|\vec{\boldsymbol{\mu}}\|_2^2 - 2\vec{\mathbf{x}}^\top \vec{\boldsymbol{\mu}} = 2(1 - \vec{\mathbf{x}}^\top \vec{\boldsymbol{\mu}})$.

5.4 FDE-based Greedy Seeding Scheme

Our goal of initialization is to quickly identify K multi-vectors from $\mathcal{D}$ as initial centroids that jointly yield a small within-cluster SNCD cost in Eq. (5) while representing distinct semantic regions of the corpus. Inspired by the local-trial variant of K-Means++, we instead grow $\mathcal{M}^{(0)}$ one multi-vector at a time. Given the centroids selected so far and a candidate multi-vector X, we evaluate the clustering cost after hypothetically adding X:

$$\sum_{i=1}^{N} \min_{M \in \mathcal{M}^{(0)} \cup \{X\}} \widehat{\mathrm{SD}}(X^{(i)}, M). \tag{14}$$

Here, we use $\widehat{\mathrm{SD}}$ computed upon cross-block FDEs (Eq. (11)) as a fast SNCD surrogate for higher efficiency. At each round, we select, among a small set of sampled candidates, the multi-vector that minimizes Eq. (14). In a nutshell, this procedure greedily chooses each new centroid according to its marginal effect on the common clustering objective. The remaining question is then how to construct a small yet informative candidate set for each round efficiently. We propose to adopt two complementary sampling rules, namely *objective-guided* and *coverage-guided* rules.

Algorithm. Specifically, Algorithm 4 shows the pseudo-code of our greedy seeding scheme. It first constructs the reusable cross-block FDEs and chooses one multi-vector X uniformly at random as the first centroid (Lines 1-2).

Afterwards, it performs $K - 1$ seeding rounds (Lines 4-10). In each round, Line 5 draws a small number of $\tau = 2 + \lfloor \ln K \rfloor$ objective-guided candidates $\mathcal{S}_{\mathrm{obj}}$ from $\mathcal{D} \setminus \mathcal{M}^{(0)}$. Particularly, at each draw, $X^{(i)}$ is sampled without replacement with probability

$$\frac{\Delta_{\mathrm{obj}}(X^{(i)}, \mathcal{M}^{(0)})}{\sum_{X^{(j)} \notin \mathcal{M}^{(0)}} \Delta_{\mathrm{obj}}(X^{(j)}, \mathcal{M}^{(0)})}, \tag{15}$$

where $\Delta_{\mathrm{obj}}(X, \mathcal{M}^{(0)}) = \min_{Y \in \mathcal{M}^{(0)}} \widehat{\mathrm{SD}}(X, Y)$ stands for the smallest distance of candidate X to current centroids $\mathcal{M}^{(0)}$. By assigning higher sampling probabilities to multi-vectors that lie far from their nearest selected centroid, this strategy encourages the initial centroids to be well dispersed across and thus broadly cover the multi-vector collection. Analogously, we additionally pick τ coverage-guided candidates $\mathcal{S}_{\mathrm{cov}}$ from $\mathcal{D} \setminus \mathcal{M}^{(0)}$ (Line 6), where each of them is sampled with probability

$$\frac{\Delta_{\mathrm{cov}}(X^{(i)}, \mathcal{M}^{(0)})}{\sum_{X^{(j)} \notin \mathcal{M}^{(0)}} \Delta_{\mathrm{cov}}(X^{(j)}, \mathcal{M}^{(0)})}, \tag{16}$$

where $\Delta_{\mathrm{cov}}(X, \mathcal{M}^{(0)}) = \min_{Y \in \mathcal{M}^{(0)}} 2 \min\{1 - \widehat{s}(X, Y), 1 - \widehat{s}(Y, X)\}$ represents the minimum *coverage distance*, i.e., the smaller of the forward and backward terms in SNCD, between candidate X and current centroids $\mathcal{M}^{(0)}$. A higher coverage distance $\Delta_{\mathrm{cov}}(X, \mathcal{M}^{(0)})$ indicates that X is far from every centroid in both directions. Intuitively, prioritizing such candidates allows the exploration of more distinct semantic regions. Next, from these two candidate pools, Algorithm 4 finds the one leading to minimal clustering cost in Eq. (14) as the new centroid (Lines 7-8). Algorithm 4 returns $\mathcal{M}^{(0)}$ after all the K centroids have been identified (Line 9).

5.5 Complexity Analysis

Let $\mathcal{D}_u$ contain the N_u multi-vectors that fail the stability test in a non-refresh iteration, and let $\widehat{N}_u = \sum_{X^{(i)} \in \mathcal{D}_u} |X^{(i)}|$. Compared with `Chamfer-KMeans`, which evaluates exact SNCD for all NK multi-vector–centroid pairs at $O(\widehat{N} K d n_{\max})$ cost, Algorithm 3 evaluates only $N + \kappa N_u$ exact pairs. The constituent-level cost is reduced to $O((\widehat{N} + \kappa \widehat{N}_u) d n_{\max})$, with the remaining work using cached distances and low-dimensional FDEs. For fixed FDE dimensions and κ, the non-refresh assignment cost becomes $O(NK + \widehat{N} d n_{\max})$, whereas a full refresh still costs $O(\widehat{N} K d n_{\max})$.

Algorithm 4 examines $O(\log K)$ candidates in each of the $K - 1$ seeding rounds. With incremental updates, initialization costs $O(NK d_f \log K)$, or $O(NK \log K)$ for fixed d_f, excluding reusable corpus-FDE construction.

Since the centroid update is unchanged, the dominant worst-case complexity of `Chamfer-KMeans+` remains $O(T \widehat{N} K d n_{\max})$. In non-refresh iterations, however, the factor K is removed from the exact SNCD term, with the remaining K-dependent work using cached or surrogate comparisons. The persistent space complexity is $O((\widehat{N} + K n_{\max}) d + NK)$. Detailed derivations are provided in the supplementary material [4].

Table 3: Statistics of the evaluated datasets.

Dataset	N	K	Avg. #Tokens
TREC	5,952	47	13.3
Reuters-52	9,100	52	86.1
20 Newsgroups	18,211	20	183.5
SciDocs-MAG	25,003	19	131.3
SciDocs-MESH	23,481	11	416.6
LoCoV1	14,838	12	5690.1
DBpedia	630,000	14	77.0
Yahoo Answers	1,460,000	10	127.4

6 Experiments

We conduct extensive experiments to evaluate multi-vector text clustering from both effectiveness and efficiency perspectives. Specifically, we compare different clustering paradigms, examine the efficiency and scalability of `Chamfer-KMeans+`, analyze its key design components, and study parameter sensitivity.

6.1 Experimental Setup

We summarize the experimental setup, including the datasets, baselines, multi-vector representations and algorithm parameters, and evaluation protocol. For brevity, detailed dataset descriptions, baseline configurations, representation and parameter settings, evaluation metric definitions, and implementation details are provided in the supplementary material [4].

Datasets. We evaluate on eight text datasets spanning different corpus sizes, numbers of ground-truth classes, domains, and text lengths. Table 3 summarizes their statistics. Our six main datasets are *TREC* [43], *Reuters-52* [34, 42], *20 Newsgroups* [34, 41], *SciDocs-MAG* [15], *SciDocs-MESH* [15], and *LoCoV1* [57]. We additionally include two substantially larger datasets, *DBpedia* [76] and *Yahoo Answers* [76], containing 630K and 1.46M texts, respectively, for large-scale evaluation. For compact presentation in figures and tables, we abbreviate *Reuters-52*, *20 Newsgroups*, *SciDocs-MAG*, *SciDocs-MESH*, and *LoCoV1* as *R-52*, *20News*, *MAG*, *MESH*, and *LoCo*, respectively.

Baselines. We compare with three groups of baselines. For *single-vector clustering*, we combine SBERT, Jina, or OpenAI embeddings with `K-Means`, `HAC`, or `SpecClust`. For *multi-vector clustering*, we consider FDE representations constructed with `MUVERA` [20] under standard clustering algorithms, as well as pairwise Wasserstein, Hausdorff, and SNCD distances with `HAC` or `SpecClust`. We also include the LLM-based methods `ClusterLLM` [77] and `k-LLMmeans` [21].

Representations and Parameters. We construct multi-vector representations from token embeddings produced by SBERT or Jina, followed by bipartite token merging (ToMe) [8] to control multi-vector cardinality. Both `Chamfer-KMeans` and `Chamfer-KMeans+` run for at most $T = 500$ iterations with convergence tolerance $\varepsilon = 10^{-4}$, and `Chamfer-KMeans+` uses candidate-set size $\kappa = 4$ by default.

Evaluation Protocol. Following convention, we adopt three widely used measures: FP [25, 64], *normalized mutual information* (NMI), and *clustering accuracy* (ACC), to assess clustering quality against the ground-truth classes of the tested datasets. In terms of efficiency and scalability evaluation, we report runtime in seconds (measured in wall-clock time), excluding the time for loading datasets, outputting clusters, embedding construction, and token merging. We omit a method if it does not finish within seven days or encounters an out-of-memory (OOM) error.

6.2 Clustering Quality Evaluation

We evaluate the clustering quality of different representation and clustering paradigms. Table 4 reports NMI, FP, and ACC on six main datasets, while Table 5 reports the results on large datasets *DBpedia* and *Yahoo Answers*. We can make the following observations.

Multi- vs. Single-Vector Clustering. Multi-vector representations provide clear effectiveness gains over strong single-vector baselines. Under `K-Means`, the best single-vector baseline achieves an average FP of 0.543, whereas `Chamfer-KMeans` and `Chamfer-KMeans+` achieve average FP scores of 0.647 and 0.649, respectively. Moreover, both methods outperform the best single-vector `K-Means` baseline in FP on each of the six main datasets. These results indicate that the effectiveness gain cannot be explained merely by replacing one single-vector encoder with another. Instead, retaining and matching multiple constituent representations provides additional semantic information that is useful for clustering.

Multi-Vector Alternatives. SNCD provides the strongest clustering quality among the generic multi-vector alternatives. With `SpecClust`, SNCD achieves an average FP of 0.586, compared with 0.530 for Wasserstein, 0.505 for Hausdorff, and 0.463 for FDE-based representations. Similarly, with `HAC`, SNCD achieves the highest average FP of 0.580, outperforming the strongest alternative, Wasserstein, at 0.541. These results show that preserving fine-grained bidirectional nearest-neighbor interactions between constituent vectors is important for effective multi-vector text clustering.

`Chamfer-KMeans` vs. `Chamfer-KMeans+`. `Chamfer-KMeans` retains the effectiveness advantage of SNCD without materializing all-pair distances. Across the six main datasets, `Chamfer-KMeans+` achieves average NMI, FP, and ACC scores of 0.598, 0.649, and 0.536, respectively, compared with 0.591, 0.647, and 0.534 for `Chamfer-KMeans`. Thus, the proposed efficiency optimizations preserve comparable clustering quality, with only small dataset-specific variations. `Chamfer-KMeans+` also outperforms the LLM-based `k-LLMmeans` baseline on average, improving FP from 0.593 to 0.649 over the six main datasets.

On the two large-scale datasets, `Chamfer-KMeans+` remains effective: it obtains NMI/FP/ACC scores of 0.854/0.887/0.854 on *DBpedia* and 0.398/0.573/0.542 on *Yahoo Answers*. It outperforms the considered scalable single-vector, FDE-based, and `k-LLMmeans` baselines while remaining close to `Chamfer-KMeans`. We next examine whether this effectiveness can be achieved with substantially lower computational cost.

6.3 Clustering Efficiency

We next evaluate the efficiency and scalability of `Chamfer-KMeans+`, by comparing it against `Chamfer-KMeans` to quantify the benefit of the proposed optimizations, and against the best baselines in Table 4 (`HAC`+SNCD and `SpecClust`+SNCD) to evaluate the advantage over pairwise multi-vector clustering baselines. We exclude `ClusterLLM` and `k-LLMmeans` from the wall-clock runtime comparison because

Table 4: Clustering quality across three method categories. Dark/light blue denotes best/runner-up results, with best overall in bold and best baseline underlined. Rankings and gains use unrounded values.

Category		Embedding or Distance	TREC			R-52			20News			MAG			MESH			LoCo			Avg.		
			NMI	FP	ACC	NMI	FP	ACC	NMI	FP	ACC	NMI	FP	ACC	NMI	FP	ACC	NMI	FP	ACC	NMI	FP	ACC
Single-Vector	SpecClust	SBERT	0.298	0.304	0.231	0.391	0.402	0.248	0.503	0.544	0.519	0.575	0.627	0.610	0.424	0.572	0.523	0.543	0.564	0.337	0.456	0.502	0.411
		Jina	0.376	0.400	0.329	0.388	0.369	0.218	0.491	0.555	0.504	0.533	0.605	0.586	0.309	0.486	0.422	0.543	0.564	0.337	0.440	0.496	0.399
		OpenAI	0.346	0.345	0.268	0.492	0.401	0.242	0.567	0.605	0.568	0.646	0.695	0.651	0.438	0.583	0.510	0.589	0.638	0.380	0.513	0.544	0.437
	HAC	SBERT	0.301	0.333	0.259	0.552	0.527	0.351	0.329	0.370	0.324	0.552	0.567	0.468	0.317	0.477	0.359	0.538	0.655	0.382	0.431	0.488	0.357
		Jina	0.267	0.272	0.183	0.522	0.534	0.378	0.404	0.426	0.304	0.497	0.487	0.363	0.161	0.420	0.260	0.401	0.571	0.304	0.375	0.452	0.299
		OpenAI	0.264	0.277	0.204	0.561	0.591	0.433	0.458	0.475	0.360	0.394	0.357	0.220	0.184	0.433	0.323	0.430	0.635	0.273	0.382	0.461	0.302
	K-Means	SBERT	0.348	0.338	0.250	0.537	0.420	0.228	0.551	0.566	0.504	0.680	0.690	0.590	0.472	0.599	0.513	0.597	0.576	0.376	0.531	0.532	0.410
		Jina	0.445	0.450	0.359	0.510	0.423	0.250	0.553	0.568	0.517	0.691	0.699	0.615	0.294	0.466	0.392	0.597	0.652	0.335	0.515	0.543	0.411
		OpenAI	0.393	0.367	0.274	0.537	0.413	0.229	0.587	0.605	0.549	0.700	0.710	0.624	0.396	0.520	0.433	0.606	0.556	0.359	0.537	0.528	0.411
LLM	ClusterLLM		0.365	0.353	0.275	0.573	0.542	0.353	0.472	0.527	0.449	0.609	0.614	0.560	0.478	0.599	0.538	0.551	0.618	0.467	0.508	0.542	0.440
	k-LLMmeans		0.466	0.466	0.376	0.554	0.440	0.250	0.564	0.593	0.544	0.702	0.745	0.697	0.520	0.674	0.602	0.605	0.641	0.437	0.568	0.593	0.485
Multi-Vector	SpecClust	FDE	0.397	0.388	0.297	0.429	0.329	0.192	0.315	0.412	0.381	0.529	0.619	0.607	0.304	0.482	0.412	0.484	0.549	0.385	0.410	0.463	0.379
		Wasserstein	0.459	0.434	0.331	0.383	0.365	0.219	0.506	0.541	0.510	0.643	0.683	0.646	0.449	0.576	0.509	0.593	0.581	0.397	0.506	0.530	0.435
		Hausdorff	0.333	0.352	0.278	0.376	0.336	0.196	0.516	0.559	0.541	0.632	0.676	0.625	0.355	0.534	0.497	0.565	0.574	0.392	0.463	0.505	0.421
		SNCD	0.491	0.480	0.387	0.561	0.445	0.276	0.558	0.597	0.581	0.677	0.726	0.697	0.461	0.599	0.536	0.639	0.668	0.396	0.565	0.586	0.479
	HAC	FDE	0.285	0.360	0.281	0.405	0.529	0.391	0.438	0.484	0.437	0.492	0.514	0.365	0.391	0.596	0.520	0.429	0.745	0.557	0.407	0.538	0.425
		Wasserstein	0.249	0.399	0.244	0.569	0.695	0.549	0.346	0.376	0.310	0.541	0.539	0.508	0.358	0.531	0.456	0.429	0.704	0.435	0.415	0.541	0.417
		Hausdorff	0.200	0.235	0.157	0.458	0.454	0.301	0.303	0.339	0.283	0.530	0.547	0.456	0.281	0.467	0.391	0.433	0.538	0.331	0.367	0.430	0.320
		SNCD	0.378	0.457	0.339	0.584	0.699	0.589	0.501	0.479	0.332	0.583	0.550	0.389	0.350	0.545	0.485	0.535	0.751	0.537	0.488	0.580	0.445
	K-Means	FDE	0.419	0.381	0.281	0.477	0.348	0.192	0.429	0.482	0.463	0.627	0.699	0.675	0.378	0.531	0.470	0.573	0.561	0.383	0.484	0.500	0.410
	Chamfer-*K*Means		0.475	0.540	0.411	0.567	0.493	0.306	0.604	0.653	0.635	0.712	0.759	0.717	0.527	0.682	0.605	0.664	0.754	0.533	0.591	0.647	0.534
	Gain vs. best baseline		↓ 0.016	↑ 0.060	↑ 0.024	↓ 0.017	↓ 0.206	↓ 0.283	↑ 0.017	↑ 0.049	↑ 0.054	↑ 0.010	↑ 0.014	↑ 0.020	↑ 0.008	↑ 0.008	↑ 0.002	↑ 0.025	↑ 0.003	↓ 0.023	↑ 0.023	↑ 0.054	↑ 0.050
	Chamfer-*K*Means+		0.493	0.534	0.415	0.558	0.495	0.311	0.618	0.676	0.631	0.706	0.755	0.725	0.534	0.681	0.611	0.678	0.751	0.523	0.598	0.649	0.536
	Gain vs. best baseline		↑ 0.003	↑ 0.054	↑ 0.028	↓ 0.027	↓ 0.204	↓ 0.278	↑ 0.031	↑ 0.071	↑ 0.050	↑ 0.005	↑ 0.010	↑ 0.028	↑ 0.014	↑ 0.008	↑ 0.009	↑ 0.039	↓< 0.001	↓ 0.034	↑ 0.030	↑ 0.056	↑ 0.052

Table 5: Clustering quality over the largest datasets.

Category	Method	*DBpedia*			*Yahoo Answers*		
		NMI	FP	ACC	NMI	FP	ACC
Single-Vector	SBERT	0.768	0.767	0.669	0.346	0.508	0.452
	Jina	0.531	0.574	0.468	0.381	0.540	0.513
LLM	*k*-LLMmeans	0.821	0.846	0.812	0.371	0.539	0.516
Multi-Vector	FDE	0.648	0.743	0.717	0.251	0.450	0.440
	Chamfer-*K*Means	**0.863**	**0.892**	0.853	0.397	0.570	0.541
	Chamfer-*K*Means+	0.854	0.887	**0.854**	**0.398**	**0.573**	**0.542**

both require external LLM API calls, whose latency depends on the model provider and service conditions and is therefore not directly comparable with local CPU/GPU execution time. Their clustering effectiveness is evaluated in Section 6.2. Figure 6 reports CPU and GPU clustering runtime across all eight datasets.

Overall Efficiency. Chamfer-*K*Means+ achieves notable speedups over multi-vector clustering baselines. Across the six datasets, it achieves a 2.7–21.5× speedup over Chamfer-*K*Means on CPU. On GPU, it consistently improves over Chamfer-*K*Means, with speedups ranging from 2.1 to 3.4×. The advantage is substantially larger relative to methods that materialize pairwise distances. Over the six datasets, Chamfer-*K*Means+ is 3.7–151.4× faster than HAC+SNCD and SpecClust+SNCD on CPU, while the GPU speedup reaches up to 40.4×. These results demonstrate the computational benefit of replacing all-pairs SNCD clustering with a centroid-based formulation and further accelerating cluster assignment.

Runtime Breakdown. Together with the Chamfer-*K*Means CPU breakdown in Figure 5(a), Figure 7 examines how the runtime composition changes after the optimizations in Chamfer-*K*Means+ on CPUs and GPUs. On CPUs, cluster assignment accounts for 94.6–97.5% of the runtime of Chamfer-*K*Means, whereas its share decreases to 55.0–89.6% in Chamfer-*K*Means+. The reduction is even more pronounced on GPUs, from 69.3–92.2% for Chamfer-*K*Means to 19.9–68.5% for Chamfer-*K*Means+. As a result, the remaining components, including centroid updates and initialization, become more prominent in the overall runtime. Overall, these results demonstrate that Chamfer-*K*Means+ effectively mitigates the assignment bottleneck identified for Chamfer-*K*Means in Section 5.1.

Scalability. The scalability advantage is more conspicuous on *DBpedia* and *Yahoo Answers*. On *DBpedia*, Chamfer-*K*Means+ achieves a 15.2× CPU speedup over Chamfer-*K*Means, while on *Yahoo Answers* it achieves a 2.4× speedup. On GPUs, the corresponding speedups are 1.7× and 2.1×, respectively. Together with the clustering-quality results in Table 5, these results show that Chamfer-*K*Means+ retains the effectiveness of fine-grained multi-vector clustering while scaling to corpora containing up to 1.46 million texts.

6.4 Ablation Study and Design Choices

We next ablate the main components of Chamfer-*K*Means+ to examine their effects on clustering quality and computational cost. Specifically, we study the stable-text pruning and FDE-based candidate retrieval in lazy cluster assignment, as well as FDE-based greedy seeding. Figure 8 reports FP and end-to-end CPU runtime on the six main datasets. Additional ablation results, including NMI, ACC, number of iterations to convergence, per-iteration assignment and centroid-update costs, component-level runtime breakdowns of ablated variants, and absolute scores for the initialization variants, are provided in the supplementary material [4].

Lazy Cluster Assignment. The two components of lazy cluster assignment substantially reduce end-to-end clustering cost while producing similar clustering quality. Disabling stable-text pruning increases total CPU runtime by 1.6–3.7× across the six datasets,

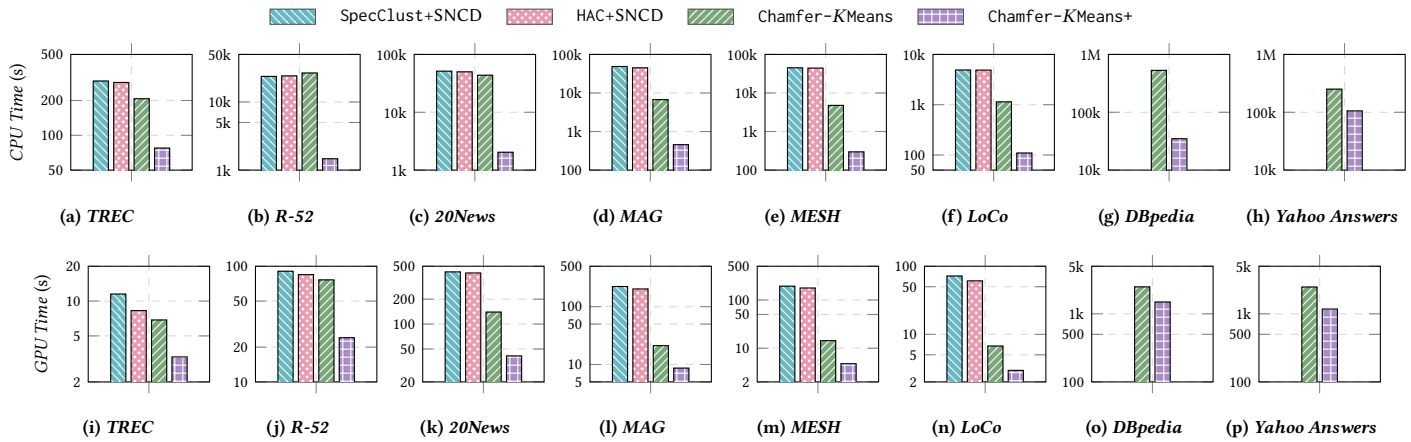


Figure 6: CPU and GPU clustering runtime of `Chamfer-KMeans`, `Chamfer-KMeans+`, and best baselines.

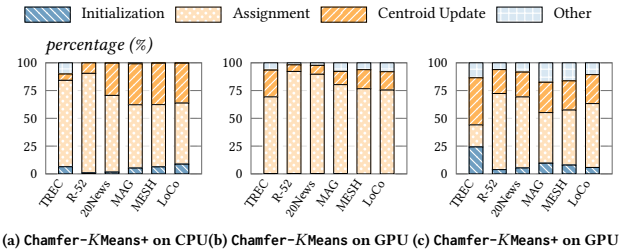


Figure 7: Runtime breakdown of `Chamfer-KMeans+` and `Chamfer-KMeans` on CPUs or GPUs. The CPU breakdown of `Chamfer-KMeans` is shown in Figure 5(a).

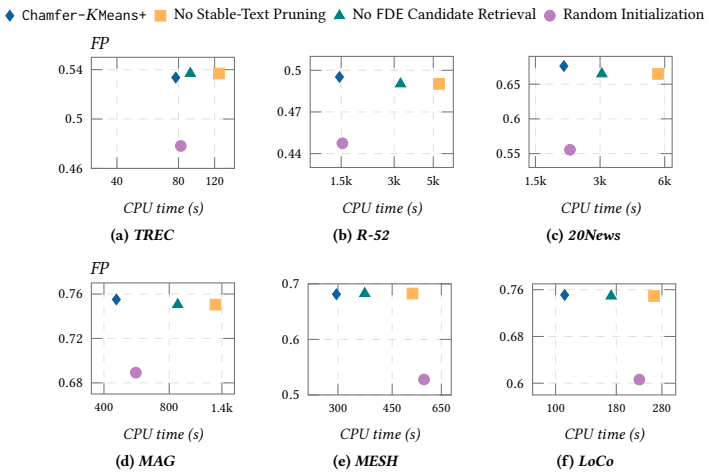


Figure 8: Clustering quality vs. runtime for the component ablation of `Chamfer-KMeans+`.

while disabling FDE-based candidate retrieval increases it by 1.2–2.2×. Meanwhile, the observed FP changes are at most 1.2 percentage points. These results confirm the complementary roles of the two components: stable-text pruning avoids exact comparisons for texts whose previous assignments can be safely certified, whereas FDE-based candidate retrieval restricts exact SNCD evaluation to a small set of promising centroids.

FDE-based Greedy Seeding. Replacing the proposed initialization with random initialization decreases FP by 4.8–15.4 percentage points and requires 1.04–2.06× as much end-to-end CPU runtime across the six datasets. These results show that the proposed greedy seeding improves both clustering quality and convergence efficiency.

Initialization Design Choices. We further examine the objective- and coverage-guided candidate generation rules. We compare the proposed design with objective-only variants using τ and 2τ local trials; the latter matches the total candidate budget of the proposed design. As shown in Figure 9, both variants underperform the proposed design, indicating that the gain is not merely due to evaluating more candidates. We also compare alternative ways of combining the two directional coverage terms. The proposed minimum rule in Eq. (16) achieves the highest NMI and FP on all six datasets, supporting its use to prioritize candidates that remain insufficiently covered in both directions.

6.5 Parameter Analysis

We finally examine three parameters of `Chamfer-KMeans+`: the Count-Sketch block dimension d_f, candidate-set size κ, and local-trial budget τ. For compact presentation, Figure 10 reports *TREC*, *20 Newsgroups*, and *LoCoV1*, which span a wide range of text lengths and cover both default block-dimension settings. Complete sensitivity results for all three parameters, together with an additional analysis of the convergence tolerance ε, are provided in the supplementary material [4].

Count-Sketch Block Dimension d_f. The effect of d_f is dataset-dependent and non-monotonic. Among the three representative datasets, the default setting is $d_f = 256$ for *TREC* and *LoCoV1*, and $d_f = 128$ for *20 Newsgroups*. Across all eight datasets, we use only the two moderate settings $d_f \in \{128, 256\}$, with each setting used for four datasets.

Candidate-Set Size κ. A small candidate set is generally sufficient, although the preferred κ varies across datasets. *20 Newsgroups*

Seeding Strategy	NMI						FP						ACC					
	TREC	*R-52*	*20News*	*MAG*	*MESH*	*LoCo*	*TREC*	*R-52*	*20News*	*MAG*	*MESH*	*LoCo*	*TREC*	*R-52*	*20News*	*MAG*	*MESH*	*LoCo*
Random Init.	−6.2	−2.5	−7.6	−2.5	−14.1	−6.3	−5.5	−4.8	−12.1	−6.6	−15.4	−14.5	−3.1	−5.8	−14.8	−13.1	−17.5	−7.8
Objective-only (τ)	−3.6	−1.7	−4.9	−1.8	−7.5	−5.8	−3.6	−7.7	−8.7	−4.5	−11.8	−13.0	−1.3	−6.7	−9.3	−5.5	−14.7	−5.8
Objective-only (2τ)	−2.1	−1.1	−4.2	−3.0	−7.3	−5.3	−2.1	−3.7	−6.8	−6.0	−9.9	−13.5	−1.4	−3.8	−3.9	−9.4	−8.8	−7.5

Coverage Combination	NMI						FP						ACC					
	TREC	*R-52*	*20News*	*MAG*	*MESH*	*LoCo*	*TREC*	*R-52*	*20News*	*MAG*	*MESH*	*LoCo*	*TREC*	*R-52*	*20News*	*MAG*	*MESH*	*LoCo*
Harmonic	−1.3	−0.8	−3.0	−1.6	−3.7	−2.8	−4.8	−3.3	−4.8	−3.6	−7.2	−10.3	−5.0	−3.6	−3.7	−5.0	−6.7	−0.8
Geometric	−1.8	−1.5	−3.7	−2.0	−4.6	−3.1	−3.1	−3.0	−6.9	−4.1	−7.2	−8.4	−2.3	−3.8	−6.8	−8.1	−7.4	+1.6
Arithmetic	−2.9	−2.1	−4.2	−2.7	−5.0	−5.1	−4.2	−6.9	−5.6	−4.9	−6.8	−10.8	−3.4	−5.8	−4.8	−7.3	−3.9	−3.8

Figure 9: Performance differences from the proposed greedy seeding scheme for alternative candidate-generation strategies and alternative rules for combining the two directional coverage terms. Each cell reports $\Delta = 100 \times$ (Variant − Proposed) percentage points.

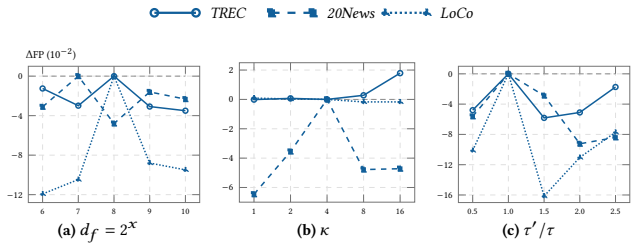


Figure 10: Parameter sensitivity of `Chamfer-KMeans+` on *TREC*, *20 Newsgroups*, and *LoCoV1*.

performs best at the default $\kappa = 4$, *LoCoV1* is nearly insensitive to κ, while *TREC* benefits from a larger candidate set. We therefore use $\kappa = 4$ as a common default rather than a dataset-specific optimum.

Local-Trial Budget τ. We vary the greedy-seeding budget from 0.5τ to 2.5τ, where the default is $\tau = 2 + \lfloor \ln K \rfloor$. The default setting achieves the highest FP on all three representative datasets, while increasing the budget does not consistently improve clustering quality. We therefore retain the default budget.

7 Related Work

7.1 Single-Vector Text Clustering

Conventional text clustering represents each document as a single vector and applies clustering algorithms such as K-`Means`, `HAC`, or spectral clustering. Early methods rely on lexical features and improve clustering through dimensionality reduction, manifold modeling, constraints, and scalable optimization [9–11, 28, 50]. Deep clustering instead learns document representations jointly with, or specifically for, clustering [32, 70–72], while more recent approaches exploit attention, contrastive learning, and pretrained language models [29, 47, 74, 75]. Despite increasingly strong representations, these methods generally compress each document into one fixed-dimensional vector, potentially obscuring fine-grained semantics.

7.2 LLM-based Text Clustering

Recent methods use LLMs either as document encoders or as semantic supervisors during clustering [38, 52]. `ClusterLLM` [77] uses LLM judgments to refine representations and clustering granularity, while related methods obtain semantic constraints or feedback from LLMs [62, 65]. k-`LLMmeans` [21] instead uses LLM-generated cluster summaries as representatives. Embedding-based approaches retain the single-vector bottleneck, whereas LLM-guided approaches require additional model inference. Our approach directly clusters compact multi-vector representations without clustering-time LLM inference.

7.3 Multi-Vector Text Representations

Multi-vector representations retain multiple constituent embeddings and have been widely studied in neural retrieval. `ColBERT` and `ColBERTv2` use contextualized token embeddings with late interaction [39, 58], while chunk-based methods preserve semantics at coarser granularities [17, 30, 44]. To reduce storage and matching costs, prior work further studies token pooling, merging, and compression [8, 14, 54].

A complementary direction accelerates multi-vector comparison. `MUVERA` [20] constructs fixed-dimensional encodings that approximate Chamfer-style similarity. These methods primarily target retrieval, whereas text clustering requires repeated comparisons with evolving set-valued centroids. Our work studies this setting under SNCD and develops dedicated exact and efficient clustering algorithms.

8 Conclusion

In this paper, we present a systematic study of multi-vector text clustering and demonstrate the effectiveness of SNCD in preserving fine-grained semantic information for clustering. Building on this finding, we develop `Chamfer-KMeans`, a Lloyd-style algorithm with a tractable closed-form centroid update and convergence guarantee, and further propose `Chamfer-KMeans+` to improve its scalability through cross-block `FDE`-based approximation, lazy cluster assignment, and `FDE`-based greedy seeding. Experiments on eight benchmark datasets show that our methods achieve strong clustering quality, while `Chamfer-KMeans+` accelerates `Chamfer-KMeans` by up to 21.5× on CPUs and scales to corpora containing 1.46 million texts, demonstrating the effectiveness and scalability of multi-vector representations for text clustering.

References

[1] Mihir Agarwal, Ankit Garg, Neeraj Kayal, Kirankumar Shiragur, et al. 2026. On Strengths and Limitations of Single-Vector Embeddings. *arXiv preprint arXiv:2603.29519* (2026).

[2] Noga Alon, Yossi Matias, and Mario Szegedy. 1996. The space complexity of approximating the frequency moments. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 20–29. https://doi.org/10.1145/237814.237823

[3] David Arthur and Sergei Vassilvitskii. 2007. k-means++ the advantages of careful seeding. In *Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms*. 1027–1035.

[4] Anonymous Author(s). 2026. Supplementary Material. https://anonymous.4open.science/r/MVTC-A34F/Supplementary.pdf.

[5] HG Barrow, JM Tenenbaum, RC Bolles, and HC Wolf. 1977. Parametric correspondence and chamfer matching: two new techniques for image matching. In *Proceedings of the 5th international joint conference on Artificial intelligence-Volume 2*. 659–663.

[6] James C Bezdek and Richard J Hathaway. 2003. Convergence of alternating optimization. *Neural, Parallel & Scientific Computations* 11, 4 (2003), 351–368.

[7] Ella Bingham and Heikki Mannila. 2001. Random projection in dimensionality reduction: applications to image and text data. In *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*. 245–250.

[8] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. 2022. Token merging: Your vit but faster. *arXiv preprint arXiv:2210.09461* (2022).

[9] Deng Cai, Xiaofei He, and Jiawei Han. 2005. Document clustering using locality preserving indexing. *IEEE transactions on knowledge and data engineering* 17, 12 (2005), 1624–1637.

[10] Deng Cai, Xiaofei He, and Jiawei Han. 2010. Locally consistent concept factorization for document clustering. *IEEE Transactions on Knowledge and Data Engineering* 23, 6 (2010), 902–913.

[11] Soumen Chakrabarti, Byron Dom, Rakesh Agrawal, and Prabhakar Raghavan. 1998. Scalable feature selection, classification and signature generation for organizing large text databases into hierarchical topic taxonomies. *The VLDB journal* 7, 3 (1998), 163–178.

[12] Chaomei Chen. 2006. CiteSpace II: Detecting and visualizing emerging trends and transient patterns in scientific literature. *Journal of the American Society for information Science and Technology* 57, 3 (2006), 359–377.

[13] Kenneth L Clarkson and David P Woodruff. 2017. Low-rank approximation and regression in input sparsity time. *Journal of the ACM (JACM)* 63, 6 (2017), 1–45.

[14] Benjamin Clavié, Antoine Chaffin, and Griffin Adams. 2024. Reducing the footprint of multi-vector retrieval with minimal performance impact via token pooling. *arXiv preprint arXiv:2409.14683* (2024).

[15] Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel S Weld. 2020. Specter: Document-level representation learning using citation-informed transformers. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 2270–2282.

[16] Marco Cuturi. 2013. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems (NIPS)*, Vol. 26. 2292–2300.

[17] Zhuyun Dai and Jamie Callan. 2019. Deeper text understanding for IR with contextual neural language modeling. In *Proceedings of the 42nd international ACM SIGIR conference on research and development in information retrieval*. 985–988.

[18] Daniel Defays. 1977. An efficient algorithm for a complete link method. *The computer journal* 20, 4 (1977), 364–366.

[19] Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[20] Laxman Dhulipala, Majid Hadian, Rajesh Jayaram, Jason Lee, and Vahab Mirrokni. 2024. Muvera: Multi-vector retrieval via fixed dimensional encoding. *Advances in Neural Information Processing Systems* 37 (2024), 101042–101073.

[21] Jairo Diaz-Rodriguez. 2026. Summaries as centroids for interpretable and scalable text clustering. In *International Conference on Learning Representations*, Vol. 2026. 43607–43630.

[22] Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. 2025. Colpali: Efficient document retrieval with vision language models. In *International Conference on Learning Representations*, Vol. 2025. 61424–61449.

[23] Zijin Feng, Luyang Lin, Lingzhi Wang, Hong Cheng, and Kam-Fai Wong. 2024. Llmedgerefine: Enhancing text clustering with llm-based boundary point refinement. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 18455–18462.

[24] Maurice Fréchet. 1948. Les éléments aléatoires de nature quelconque dans un espace distancié. In *Annales de l'institut Henri Poincaré*, Vol. 10. 215–310.

[25] Jiajie Fu, Haitong Tang, Arijit Khan, Sharad Mehrotra, Xiangyu Ke, and Yunjun Gao. 2025. In-context clustering-based entity resolution with large language models: A design space exploration. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–28.

[26] Hang Gao, Wujiang Xu, Kai Mei, and Dimitris N Metaxas. 2026. Pooling and Semantic Shift: The Fundamental Challenges in Long Text Embedding and Retrieval. *arXiv preprint arXiv:2603.21437* (2026).

[27] Kishen N Gowda, Willem Fletcher, MohammadHossein Bateni, Laxman Dhulipala, D Ellis Hershkowitz, Rajesh Jayaram, and Jakub Łącki. 2026. Chamfer-Linkage for Hierarchical Agglomerative Clustering. *arXiv preprint arXiv:2602.10444* (2026).

[28] Renchu Guan, Xiaohu Shi, Maurizio Marchese, Chen Yang, and Yanchun Liang. 2010. Text clustering with seeds affinity propagation. *IEEE Transactions on Knowledge and Data Engineering* 23, 4 (2010), 627–637.

[29] Renchu Guan, Hao Zhang, Yanchun Liang, Fausto Giunchiglia, Lan Huang, and Xiaoyue Feng. 2020. Deep feature-based text clustering and its explanation. *IEEE Transactions on Knowledge and Data Engineering* 34, 8 (2020), 3669–3680.

[30] Michael Günther, Isabelle Mohr, Daniel James Williams, Bo Wang, and Han Xiao. 2024. Late chunking: Contextual chunk embeddings using long-context embedding models. *arXiv preprint arXiv:2409.04701* (2024).

[31] Michael Günther, Saba Sturua, Mohammad Kalim Akram, Isabelle Mohr, Andrei Ungureanu, Bo Wang, Sedigheh Eslami, Scott Martens, Maximilian Werk, Nan Wang, et al. 2025. jina-embeddings-v4: Universal embeddings for multimodal multilingual retrieval. In *Proceedings of the 5th Workshop on Multilingual Representation Learning (MRL 2025)*. 531–550.

[32] Xifeng Guo, Long Gao, Xinwang Liu, and Jianping Yin. 2017. Improved deep embedded clustering with local structure preservation.. In *Ijcai*, Vol. 17. 1753–1759.

[33] John A Hartigan and Manchek A Wong. 1979. Algorithm AS 136: A k-means clustering algorithm. *Journal of the royal statistical society. series c (applied statistics)* 28, 1 (1979), 100–108.

[34] Dan Hendrycks and Kevin Gimpel. 2016. A baseline for detecting misclassified and out-of-distribution examples in neural networks. *arXiv preprint arXiv:1610.02136* (2016).

[35] Fu-Chiang Hsu, Amy JC Trappey, Charles V Trappey, Jiang-Liang Hou, and Shang-Jyh Liu. 2006. Technology and knowledge document cluster analysis for enterprise R&D strategic planning. *International Journal of Technology Management* 36, 4 (2006), 336–353.

[36] Daniel P Huttenlocher, Gregory A. Klanderman, and William J Rucklidge. 2002. Comparing images using the Hausdorff distance. *IEEE Transactions on pattern analysis and machine intelligence* 15, 9 (2002), 850–863.

[37] Rajesh Jayaram. 2026. Multi-Vector Embeddings are Provably More Expressive than Single Vector Embeddings. *arXiv preprint arXiv:2606.23475* (2026).

[38] Imed Keraghel, Stanislas Morbieu, and Mohamed Nadif. 2024. Beyond words: a comparative analysis of LLM embeddings for effective clustering. In *International symposium on intelligent data analysis*. Springer, 205–216.

[39] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 39–48.

[40] Lasse Kohlmeyer, Tim Repke, and Ralf Krestel. 2021. Novel views on novels: Embedding multiple facets of long texts. In *IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology*. 670–675.

[41] Ken Lang. 1995. Newsweeder: Learning to filter netnews. In *Machine learning proceedings 1995*. Elsevier, 331–339.

[42] David Lewis. 1997. Reuters-21578 text categorization test collection. *Distribution 1.0, AT&T Labs-Research* (1997).

[43] Xin Li and Dan Roth. 2002. Learning question classifiers. In *Coling 2002: The 19th international conference on computational linguistics*.

[44] Robert Litschko, Ivan Vulić, Simone Paolo Ponzetto, and Goran Glavaš. 2022. On cross-lingual retrieval with multilingual text encoders. *Information Retrieval Journal* 25, 2 (2022), 149–183.

[45] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).

[46] Stuart Lloyd. 1982. Least squares quantization in PCM. *IEEE transactions on information theory* 28, 2 (1982), 129–137.

[47] Minkuan Lu, Jianhua Yin, Kaijun Wang, and Liqiang Nie. 2024. A multi-view clustering algorithm for short text. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 5101–5110.

[48] Steinbach Michael. 2000. A comparison of document clustering techniques. In *KDD Workshop on Text Mining, 2000*.

[49] Fionn Murtagh and Pedro Contreras. 2012. Algorithms for hierarchical clustering: an overview. *Wiley interdisciplinary reviews: data mining and knowledge discovery* 2, 1 (2012), 86–97.

[50] Odysseas Papapetrou, Wolf Siberski, and Norbert Fuhr. 2011. Decentralized probabilistic text clustering. *IEEE Transactions on Knowledge and Data Engineering* 24, 10 (2011), 1848–1861.

[51] Anup Pattnaik, Cijo George, Rishabh Kumar Tripathi, Sasanka Vutla, and Jithendra Vepa. 2024. Improving hierarchical text clustering with llm-guided multi-view

cluster representation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. 719–727.

[52] Alina Petukhova, João P Matos-Carvalho, and Nuno Fachada. 2025. Text clustering with large language model embeddings. *International Journal of Cognitive Computing in Engineering* 6 (2025), 100–108.

[53] Alexandrin Popescul, Gary William Flake, Steve Lawrence, Lyle H Ungar, and C Lee Giles. 2000. Clustering and identifying temporal trends in document databases. In *Proceedings IEEE Advances in Digital Libraries 2000*. IEEE, 173–182.

[54] Hanxiang Qin, Alexander Martin, Rohan Jha, Chunsheng Zuo, Reno Kriz, and Benjamin Van Durme. 2026. Multi-vector index compression in any modality. *arXiv preprint arXiv:2602.21202* (2026).

[55] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 3982–3992.

[56] Yossi Rubner, Carlo Tomasi, and Leonidas J Guibas. 1998. A metric for distributions with applications to image databases. In *Sixth international conference on computer vision (IEEE Cat. No. 98CH36271)*. IEEE, 59–66.

[57] Jon Saad-Falcon, Daniel Y Fu, Simran Arora, Neel Guha, and Christopher Ré. 2024. Benchmarking and building long-context retrieval models with loco and m2-bert. *arXiv preprint arXiv:2402.07440* (2024).

[58] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. Colbertv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 3715–3734.

[59] Robin Sibson. 1973. SLINK: an optimally efficient algorithm for the single-link cluster method. *The computer journal* 16, 1 (1973), 30–34.

[60] Sandeep Silwal, Sara Ahmadian, Andrew Nystrom, Andrew McCallum, Deepak Ramachandran, and Mehran Kazemi. 2023. Kwikbucks: Correlation clustering with cheap-weak and expensive-strong signals. In *Proceedings of The Fourth Workshop on Simple and Efficient Natural Language Processing (SustaiNLP)*. 1–31.

[61] Lovro Šubelj, Nees Jan Van Eck, and Ludo Waltman. 2016. Clustering scientific publications based on citation relations: A systematic comparison of different methods. *PloS one* 11, 4 (2016), e0154404.

[62] Vijay Viswanathan, Kiril Gashteovski, Kiril Gashteovski, Carolin Lawrence, Tongshuang Wu, and Graham Neubig. 2024. Large language models enable few-shot clustering. *Transactions of the Association for Computational Linguistics* 12 (2024), 321–333.

[63] Ulrike Von Luxburg. 2007. A tutorial on spectral clustering. *Statistics and computing* 17, 4 (2007), 395–416.

[64] Hongtao Wang, Renchi Yang, Haoran Zheng, and Xiangyu Ke. 2026. Adaptive Graph Refinement and Label Propagation with LLMs for Cost-Effective Entity Resolution. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 4906–4915.

[65] Hongtao Wang, Taiyan Zhang, Renchi Yang, and Jianliang Xu. 2025. Cequel: Cost-Effective Querying of Large Language Models for Text Clustering. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 2998–3008.

[66] Yongheng Wang, Yan Jia, and Shuqiang Yang. 2006. Short documents clustering in very large text databases. In *International Conference on Web Information Systems Engineering*. Springer, 83–93.

[67] Zihao Wang, Hang Yin, Lihui Liu, Hanghang Tong, Yangqiu Song, Ginny Wong, and Simon See. 2026. R2k is theoretically large enough for embedding-based top-k retrieval. *arXiv preprint arXiv:2601.20844* (2026).

[68] Joe H Ward Jr. 1963. Hierarchical grouping to optimize an objective function. *Journal of the American statistical association* 58, 301 (1963), 236–244.

[69] Orion Weller, Michael Boratko, Iftekhar Naim, and Jinhyuk Lee. 2026. On the theoretical limitations of embedding-based retrieval. In *International Conference on Learning Representations*, Vol. 2026. 11202–11216.

[70] Junyuan Xie, Ross Girshick, and Ali Farhadi. 2016. Unsupervised deep embedding for clustering analysis. In *International conference on machine learning*. PMLR, 478–487.

[71] Jiaming Xu, Peng Wang, Guanhua Tian, Bo Xu, Jun Zhao, Fangyuan Wang, and Hongwei Hao. 2015. Short text clustering via convolutional neural networks. In *Proceedings of the 1st workshop on vector space modeling for natural language processing*. 62–69.

[72] Jiaming Xu, Bo Xu, Peng Wang, Suncong Zheng, Guanhua Tian, and Jun Zhao. 2017. Self-taught convolutional neural networks for short text clustering. *Neural Networks* 88 (2017), 22–31.

[73] Charles Zhang, Benji Peng, Xintian Sun, Qian Niu, Junyu Liu, Keyu Chen, Ming Li, Pohsun Feng, Ziqian Bi, Ming Liu, et al. 2024. From word vectors to multimodal embeddings: Techniques, applications, and future directions for large language models. *arXiv preprint arXiv:2411.05036* (2024).

[74] Dejiao Zhang, Feng Nan, Xiaokai Wei, Shang-Wen Li, Henghui Zhu, Kathleen McKeown, Ramesh Nallapati, Andrew O Arnold, and Bing Xiang. 2021. Supporting clustering with contrastive learning. In *Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: Human language technologies*. 5419–5430.

[75] Wei Zhang, Chao Dong, Jianhua Yin, and Jianyong Wang. 2021. Attentive representation learning with adversarial training for short text clustering. *IEEE Transactions on Knowledge and Data Engineering* 34, 11 (2021), 5196–5210.

[76] Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level convolutional networks for text classification. *Advances in neural information processing systems* 28 (2015).

[77] Yuwei Zhang, Zihan Wang, and Jingbo Shang. 2023. Clusterllm: Large language models as a guide for text clustering. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 13903–13920.

[78] Dongfang Zhao. 2025. Approximate Hausdorff Distance for Multi-Vector Databases. *arXiv preprint arXiv:2503.06833* (2025).